

PROIECT PARTEA A 2-A

Andronescu Mihai
Alexandru



Descriere:

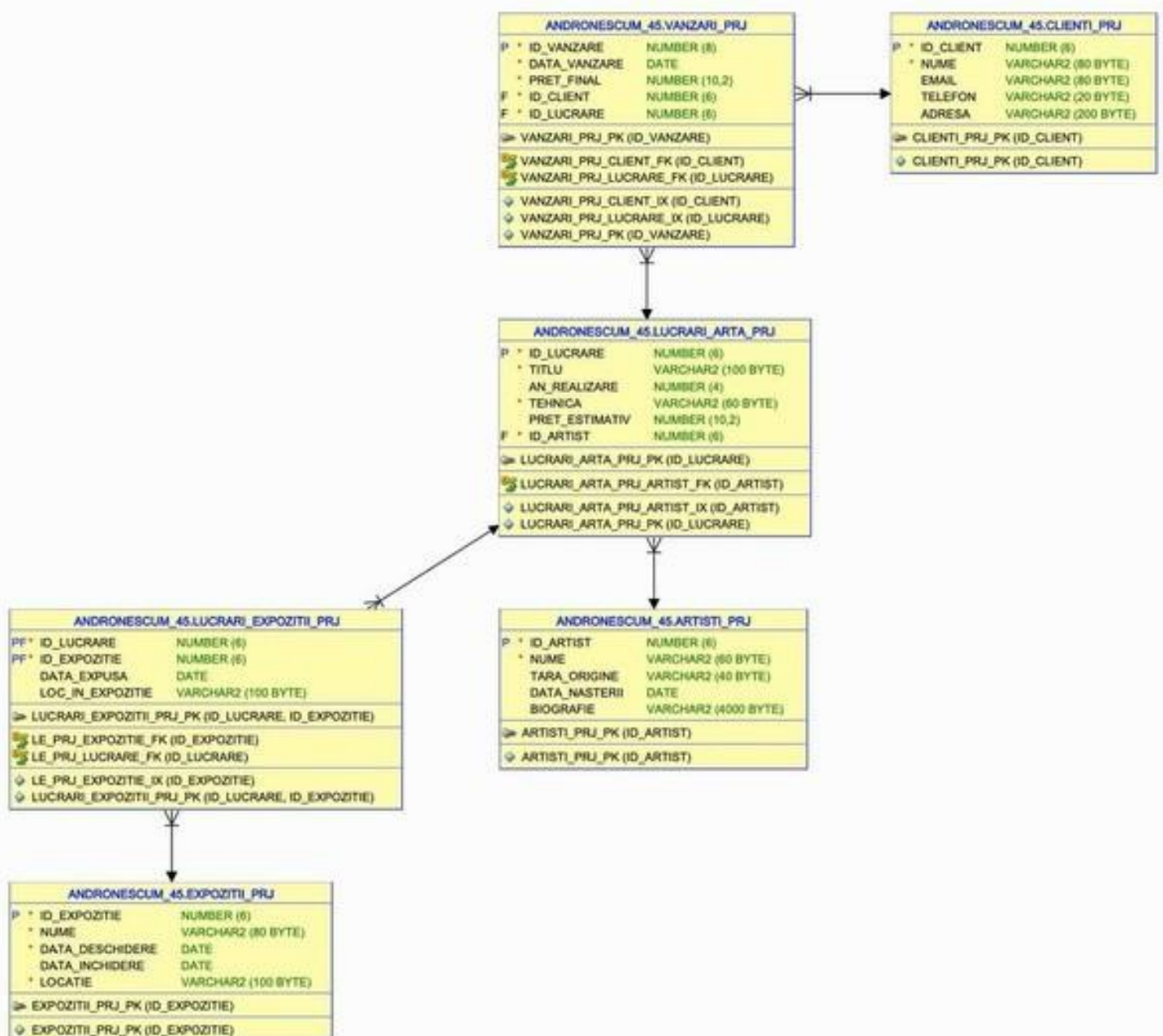
Scopul acestui proiect este dezvoltarea unei baze de date pentru o galerie de artă care gestionează artiști, lucrări, expoziții și vânzări. Baza de date trebuie să permită stocarea și organizarea informațiilor esențiale, astfel încât activitatea galeriei să poată fi urmărită ușor și structurat.

Galeria colaborează cu mai mulți artiști, fiecare având un ID unic, nume, țară de origine, data nașterii și o scurtă descriere artistică. Fiecare artist poate crea mai multe lucrări de artă, iar pentru fiecare lucrare se memorează titlul, anul realizării, tehnica folosită și un preț estimativ. O lucrare aparține unui singur artist, dar poate participa la mai multe expoziții și poate fi vândută către diferiți clienți.

Expozițiile organizate de galerie au nume, perioadă de desfășurare și locație. În cadrul unei expoziții pot fi expuse mai multe lucrări, iar o lucrare poate apărea în mai multe expoziții. Pentru a reprezenta această relație mulți-la-mulți, un tabel intermediar reține atât lucrarea, cât și expoziția, împreună cu data expunerii și locul pe perete

Clienții sunt de asemenea gestionați în baza de date, fiind identificați printr-un ID unic și având informații de contact precum nume, email, telefon și adresă. Când un client cumpără o lucrare, se înregistrează o vânzare care memorează data tranzacției, prețul final, lucrarea și clientul.

Prin această structură, baza de date permite interogări utile în activitatea galeriei, precum lista lucrărilor expuse într-o expoziție, clienții care au făcut cele mai multe achiziții sau lucrările celor mai vânduți artiști. Astfel, sistemul oferă suport atât operațional, cât și pentru analiză și decizii de management.



Prima Parte:

Structuri de control:

1) Să se creeze un bloc PL/SQL care citește ID_VANZARE de la tastatură și afișează informații despre vânzare și lucrarea asociată. Să se compare pret_final cu pret_estimativ, iar dacă pret_final este mai mic decât 90% din estimare să se majoreze cu 5%, iar dacă este mai mare decât 120% să se reducă cu 3%. La final să se afișeze valorile inițiale și cele modificate ale prețului. cod:

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    v_id_vanzare VANZARI_PRJ.id_vanzare%TYPE := &id_vanzare;
```

```
    v_id_lucrare VANZARI_PRJ.id_lucrare%TYPE;
```

```
    v_pret_final VANZARI_PRJ.pret_final%TYPE;
```

```
    v_titlu      LUCRARI_ARTA_PRJ.titlu%TYPE;
```

```
    v_pret_est   LUCRARI_ARTA_PRJ.pret_estimativ%TYPE;
```

```
    v_nou_final  NUMBER;
```

```
    v_status     VARCHAR2(20);
```

```
BEGIN
```

```
    SELECT id_lucrare, pret_final
```

```
    INTO v_id_lucrare, v_pret_final
```

```
    FROM VANZARI_PRJ
```

```
    WHERE id_vanzare = v_id_vanzare;
```

```
SELECT titlu, pret_estimativ  
INTO v_titlu, v_pret_est  
FROM LUCRARI_ARTA_PRJ  
WHERE id_lucrare = v_id_lucrare;
```

```
v_nou_final := v_pret_final;
```

```
v_status := 'OK';
```

```
IF v_pret_final < 0.9 * v_pret_est THEN
```

```
    v_status := 'DISCOUNT';
```

```
    v_nou_final := ROUND(v_pret_final * 1.05, 2);
```

```
ELSIF v_pret_final > 1.2 * v_pret_est THEN
```

```
    v_status := 'PREMIUM';
```

```
    v_nou_final := ROUND(v_pret_final * 0.97, 2);
```

```
END IF;
```

```
UPDATE VANZARI_PRJ
```

```
SET pret_final = v_nou_final
```

```
WHERE id_vanzare = v_id_vanzare
```

```
RETURNING pret_final INTO v_nou_final;
```

```
DBMS_OUTPUT.PUT_LINE('ID_VANZARE | TITLU | ESTIMAT | FINAL_OLD | FINAL_NEW |  
STATUS');
```

```
DBMS_OUTPUT.PUT_LINE(
```

```
    v_id_vanzare || ' ' || v_titlu || ' ' || v_pret_est || ' ' ||
```

```
    v_pret_final || ' ' || v_nou_final || ' ' || v_status
```

```
);
```

COMMIT;

END;

/

```
Worksheet Query Builder

SET SERVEROUTPUT ON;

DECLARE
    v_id_vanzare VANZARI_PRJ.id_vanzare%TYPE := &id_vanzare;

    v_id_lucrare VANZARI_PRJ.id_lucrare%TYPE;
    v_pret_final VANZARI_PRJ.pret_final%TYPE;

    v_titlu LUCRARI_ARTA_PRJ.titlu%TYPE;
    v_pret_est LUCRARI_ARTA_PRJ.pret_estimativ%TYPE;

    v_nou_final NUMBER;
    v_status VARCHAR2(20);
BEGIN
    SELECT id_lucrare, pret_final
    INTO v_id_lucrare, v_pret_final
    FROM VANZARI_PRJ
    WHERE id_vanzare = v_id_vanzare;

    SELECT titlu, pret_estimativ
    INTO v_titlu, v_pret_est
    FROM LUCRARI_ARTA_PRJ
    WHERE id_lucrare = v_id_lucrare;

    v_nou_final := v_pret_final;
    v_status := 'OK';

    IF v_pret_final < 0.9 * v_pret_est THEN
        v_status := 'DISCOUNT';
        v_nou_final := ROUND(v_pret_final * 1.05, 2);
    ELSIF v_pret_final > 1.2 * v_pret_est THEN
        v_status := 'PREMIUM';
        v_nou_final := ROUND(v_pret_final * 0.97, 2);
    END IF;

    UPDATE VANZARI_PRJ
    SET pret_final = v_nou_final
    WHERE id_vanzare = v_id_vanzare
    RETURNING pret_final INTO v_nou_final;

    DBMS_OUTPUT.PUT_LINE('ID_VANZARE | TITLU | ESTIMAT | FINAL_OLD | FINAL_NEW | STATUS');
    DBMS_OUTPUT.PUT_LINE(
        v_id_vanzare || ' | ' || v_titlu || ' | ' || v_pret_est || ' | ' ||
        v_pret_final || ' | ' || v_nou_final || ' | ' || v_status
    );

    COMMIT;
END;
/
```

ID_VANZARE	TITLU	ESTIMAT	FINAL_OLD	FINAL_NEW	STATUS
4	Abstractie albastra	2860	2700	2700	OK

PL/SQL procedure successfully completed.

2) Să se creeze un bloc PL/SQL care citește de la tastatură ID_CLIENT și afișează informații despre clientul respectiv, împreună cu numărul total de vânzări realizate și suma totală cheltuită. În funcție de valoarea totală a cumpărăturilor, clientul va fi încadrat în categoria VIP dacă totalul este de cel puțin 8000, respectiv NORMAL în caz contrar. La final să se afișeze toate informațiile obținute. cod:

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    v_id_client  CLIENTI_PRJ.id_client%TYPE := &id_client;
```

```
    v_nume       CLIENTI_PRJ.numename%TYPE;
```

```
    v_email      CLIENTI_PRJ.email%TYPE;
```

```
    v_nr_vanzari NUMBER;
```

```
    v_total      NUMBER;
```

```
    v_categorie  VARCHAR2(20);
```

```
BEGIN
```

```
    SELECT nume, email
```

```
    INTO v_nume, v_email
```

```
    FROM CLIENTI_PRJ
```

```
    WHERE id_client = v_id_client;
```

```
    SELECT COUNT(*), NVL(SUM(pret_final), 0)
```

```
    INTO v_nr_vanzari, v_total
```

```
    FROM VANZARI_PRJ
```

```
    WHERE id_client = v_id_client;
```

```
    IF v_total >= 8000 THEN
```

```

        v_categorie := 'VIP';

ELSE

        v_categorie := 'NORMAL';

END IF;

DBMS_OUTPUT.PUT_LINE('ID_CLIENT | NUME | EMAIL | NR_VANZARI | TOTAL |
CATEGORIE');

DBMS_OUTPUT.PUT_LINE(v_id_client || ' | ' || v_nume || ' | ' || v_email || ' | ' ||
        v_nr_vanzari || ' | ' || v_total || ' | ' || v_categorie);

END;

/

```

```

        v_nr_vanzari || ' | ' || v_total || ' | ' || v_categorie);
END;
ID_CLIENT | NUME | EMAIL | NR_VANZARI | TOTAL | CATEGORIE
3 | Ana Marinescu | ana.m@galerie.ro | 2 | 6200 | NORMAL

PL/SQL procedure successfully completed.

```

3) Să se creeze un bloc PL/SQL care citește ID_EXPOZITIE și afișează informații despre expoziția respectivă, precum și numărul total de lucrări expuse în cadrul acesteia. În plus, să se verifice dacă expoziția este IN DESFASURARE sau INCHEIATA, comparând data curentă cu perioada de desfășurare a expoziției. La final să se afișeze toate informațiile obținute. cod:

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```

v_id_expozitie EXPOZITII_PRJ.id_expozitie%TYPE := &id_expozitie;

v_nume        EXPOZITII_PRJ.numa%TYPE;

v_locatie     EXPOZITII_PRJ.locatie%TYPE;

v_data_desch  EXPOZITII_PRJ.data_deschidere%TYPE;

v_data_inch   EXPOZITII_PRJ.data_inchidere%TYPE;

v_nr_lucrari  NUMBER;

```

```

v_status    VARCHAR2(20);

BEGIN

    SELECT nume, locatie, data_deschidere, data_inchidere
    INTO v_nume, v_locatie, v_data_desch, v_data_inch
    FROM EXPOZITII_PRJ
    WHERE id_expozitie = v_id_expozitie;

    SELECT COUNT(*)
    INTO v_nr_lucrari
    FROM LUCRARI_EXPOZITII_PRJ
    WHERE id_expozitie = v_id_expozitie;

    IF SYSDATE BETWEEN v_data_desch AND v_data_inch THEN
        v_status := 'IN DESFASURARE';
    ELSE
        v_status := 'INCHEIATA';
    END IF;

    DBMS_OUTPUT.PUT_LINE('ID_EXPOZITIE | NUME | LOCATIE | DATA_DESCH |
DATA_INCH | NR_LUCRARI | STATUS');

    DBMS_OUTPUT.PUT_LINE(v_id_expozitie || ' | ' || v_nume || ' | ' || v_locatie || ' | ' ||
        TO_CHAR(v_data_desch, 'DD-MM-YYYY') || ' | ' ||
        TO_CHAR(v_data_inch, 'DD-MM-YYYY') || ' | ' ||
        v_nr_lucrari || ' | ' || v_status);

END;

/

```



```
Script Output x
Task completed in 2,981 seconds

v_data_inch      EXPOZITII_PRJ.data_inchidere%TYPE;
v_nr_lucrari     NUMBER;
v_status         VARCHAR2(20);
BEGIN
  SELECT nume, locatie, data_deschidere, data_inchidere
  INTO v_nume, v_locatie, v_data_desch, v_data_inch
  FROM EXPOZITII_PRJ
  WHERE id_expozitie = v_id_expozitie;

  SELECT COUNT(*)
  INTO v_nr_lucrari
  FROM LUCRARI_EXPOZITII_PRJ
  WHERE id_expozitie = v_id_expozitie;

  IF SYSDATE BETWEEN v_data_desch AND v_data_inch THEN
    v_status := 'IN DESFASURARE';
  ELSE
    v_status := 'INCHEIATA';
  END IF;

  DBMS_OUTPUT.PUT_LINE('ID_EXPOZITIE | NUME | LOCATIE | DATA_DESCH | DATA_INCH | NR_LUCI
  DBMS_OUTPUT.PUT_LINE(v_id_expozitie || ' | ' || v_nume || ' | ' || v_locatie || ' | '
                        TO_CHAR(v_data_desch, 'DD-MM-YYYY') || ' | ' ||
                        TO_CHAR(v_data_inch, 'DD-MM-YYYY') || ' | ' ||
                        v_nr_lucrari || ' | ' || v_status);
END;
ID_EXPOZITIE | NUME | LOCATIE | DATA_DESCH | DATA_INCH | NR_LUCRARI | STATUS
2 | Abstract si Forme | Galeria ArtSpace, Cluj | 05-04-2023 | 20-04-2023 | 6 | INCHEIATA

PL/SQL procedure successfully completed.
```

4) Să se creeze un bloc PL/SQL care citește ID_ARTIST și actualizează prețul estimativ al lucrărilor realizate de acel artist. Dacă tehnica este ULEI prețul estimativ se majorează cu 15%, dacă tehnica este ACRILIC se majorează cu 10%, iar pentru celelalte tehnici se majorează cu 5%. La final să se afișeze titlul lucrării și noul preț estimativ. cod:

SET SERVEROUTPUT ON;

DECLARE

v_id_artist LUCRARI_ARTA_PRJ.id_artist%TYPE := &id_artist;

v_min_id NUMBER;

v_max_id NUMBER;

v_exista NUMBER;

```
v_tehnica LUCRARI_ARTA_PRJ.tehnica%TYPE;
```

```
v_titlu LUCRARI_ARTA_PRJ.titlu%TYPE;
```

```
v_old NUMBER;
```

```
v_new NUMBER;
```

```
v_modificate NUMBER := 0;
```

```
BEGIN
```

```
SELECT MIN(id_lucrare), MAX(id_lucrare)
```

```
INTO v_min_id, v_max_id
```

```
FROM LUCRARI_ARTA_PRJ
```

```
WHERE id_artist = v_id_artist;
```

```
DBMS_OUTPUT.PUT_LINE('ID | TITLU | TEHNICA | OLD | NEW');
```

```
FOR v_id IN v_min_id..v_max_id LOOP
```

```
    SELECT COUNT(*)
```

```
    INTO v_exista
```

```
    FROM LUCRARI_ARTA_PRJ
```

```
    WHERE id_lucrare = v_id
```

```
    AND id_artist = v_id_artist;
```

```
IF v_exista = 1 THEN
```

```
    SELECT titlu, tehnica, pret_estimativ
```

```
    INTO v_titlu, v_tehnica, v_old
```

```
    FROM LUCRARI_ARTA_PRJ
```

WHERE id_lucrare = v_id;

v_new := v_old;

IF UPPER(v_tehnica) = 'ULEI' THEN

 v_new := ROUND(v_old * 1.15, 2);

ELSIF UPPER(v_tehnica) = 'ACRILIC' THEN

 v_new := ROUND(v_old * 1.10, 2);

ELSE

 v_new := ROUND(v_old * 1.05, 2);

END IF;

UPDATE LUCRARI_ARTA_PRJ

SET pret_estimativ = v_new

WHERE id_lucrare = v_id;

v_modifycate := v_modifycate + 1;

DBMS_OUTPUT.PUT_LINE(v_id || ' ' || v_titlu || ' ' || v_tehnica || ' ' || v_old
|| ' ' || v_new);

END IF;

END LOOP;

DBMS_OUTPUT.PUT_LINE('TOTAL MODIFICATE: ' || v_modifycate);

COMMIT;

END;

/

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    v_id_artist    LUCRARI_ARTA_PRJ.id_artist%TYPE := &id_artist;  
    v_min_id       NUMBER;  
    v_max_id       NUMBER;
```

```
    v_exista       NUMBER;  
    v_tehnica      LUCRARI_ARTA_PRJ.tehnica%TYPE;  
    v_titlu        LUCRARI_ARTA_PRJ.titlu%TYPE;  
    v_old          NUMBER;  
    v_new          NUMBER;
```

```
    v_modificate   NUMBER := 0;
```

```
BEGIN
```

```
    SELECT MIN(id_lucrare), MAX(id_lucrare)  
    INTO v_min_id, v_max_id  
    FROM LUCRARI_ARTA_PRJ  
    WHERE id_artist = v_id_artist;
```

```
    DBMS_OUTPUT.PUT_LINE('ID | TITLU | TEHNICA | OLD | NEW');
```

```
    FOR v_id IN v_min_id..v_max_id LOOP
```

```
        SELECT COUNT(*)  
        INTO v_exista  
        FROM LUCRARI_ARTA_PRJ  
        WHERE id_lucrare = v_id  
               AND id_artist = v_id_artist;
```

```
        IF v_exista = 1 THEN
```

```
            SELECT titlu, tehnica, pret_estimativ  
            INTO v_titlu, v_tehnica, v_old  
            FROM LUCRARI_ARTA_PRJ  
            WHERE id_lucrare = v_id;
```

```
            v_new := v_old;
```

```
            IF UPPER(v_tehnica) = 'ULEI' THEN  
                v_new := ROUND(v_old * 1.15, 2);  
            ELSIF UPPER(v_tehnica) = 'ACRILIC' THEN  
                v_new := ROUND(v_old * 1.10, 2);  
            ELSE  
                v_new := ROUND(v_old * 1.05, 2);  
            END IF;
```

```
            UPDATE LUCRARI_ARTA_PRJ  
            SET pret_estimativ = v_new  
            WHERE id_lucrare = v_id;
```

```
            v_modificate := v_modificate + 1;
```

```
            DBMS_OUTPUT.PUT_LINE(v_id || ' | ' || v_titlu || ' | ' || v_tehnica || ' | ' || v_old || ' | ' || v_new);
```

```
        END IF;
```

```
    END LOOP;
```

```
    DBMS_OUTPUT.PUT_LINE('TOTAL MODIFICATE: ' || v_modificate);
```

```
    COMMIT;
```

```
END;
```

ID	TITLU	TEHNICA	OLD	NEW
20	Instalatie luminoasa	Instalatie	4750	4987,5
21	Umbre in oras	Ulei pe panza	3600	3780
22	Texturi urbane	Mixed media	3400	3570

TOTAL MODIFICATE: 3

PL/SQL procedure successfully completed.

5) Să se creeze un bloc PL/SQL care citește ID_CLIENT și afișează un raport privind activitatea clientului. Să se determine numărul total de vânzări și suma totală cheltuită, iar clientul să fie clasificat ca VIP dacă totalul depășește un prag stabilit sau NORMAL în caz contrar. De asemenea, să se afișeze lucrarea achiziționată la cel mai mare preț. cod:

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    v_id_client CLIENTI_PRJ.id_client%TYPE := &id_client;
```

```
    v_nume      CLIENTI_PRJ.nume%TYPE;
```

```
    v_nr        NUMBER;
```

```
    v_total     NUMBER;
```

```
    v_id_v_max  VANZARI_PRJ.id_vanzare%TYPE;
```

```
    v_p_max     VANZARI_PRJ.pret_final%TYPE;
```

```
    v_id_luc    VANZARI_PRJ.id_lucrare%TYPE;
```

```
    v_titlu     LUCRARI_ARTA_PRJ.titlu%TYPE;
```

```
BEGIN
```

```
    SELECT nume INTO v_nume
```

```
    FROM CLIENTI_PRJ
```

WHERE id_client = v_id_client;

SELECT COUNT(*), NVL(SUM(pret_final),0)

INTO v_nr, v_total

FROM VANZARI_PRJ

WHERE id_client = v_id_client;

DBMS_OUTPUT.PUT_LINE('CLIENT | NR_VANZARI | TOTAL | CATEGORIE');

IF v_total >= 8000 THEN

DBMS_OUTPUT.PUT_LINE(v_nume || ' | ' || v_nr || ' | ' || v_total || ' | VIP');

ELSE

DBMS_OUTPUT.PUT_LINE(v_nume || ' | ' || v_nr || ' | ' || v_total || ' | NORMAL');

END IF;

IF v_nr > 0 THEN

SELECT id_vanzare, pret_final, id_lucrare

INTO v_id_v_max, v_p_max, v_id_luc

FROM VANZARI_PRJ

WHERE id_client = v_id_client

AND pret_final = (SELECT MAX(pret_final) FROM VANZARI_PRJ WHERE id_client =
v_id_client)

FETCH FIRST 1 ROW ONLY;

SELECT titlu INTO v_titlu

FROM LUCRARI_ARTA_PRJ

WHERE id_lucrare = v_id_luc;

```
        DBMS_OUTPUT.PUT_LINE('TOP: vanzare ' || v_id_v_max || ' | ' || v_p_max || ' |  
lucrare: ' || v_titlu);
```

```
    ELSE
```

```
        DBMS_OUTPUT.PUT_LINE('TOP: clientul nu are achizitii.');
```

```
    END IF;
```

```
END;
```

```
/
```

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    v_id_client CLIENTI_PRJ.id_client%TYPE := &id_client;  
    v_num     CLIENTI_PRJ.num%TYPE;
```

```
    v_nr      NUMBER;  
    v_total   NUMBER;
```

```
    v_id_v_max VANZARI_PRJ.id_vanzare%TYPE;  
    v_p_max    VANZARI_PRJ.pret_final%TYPE;  
    v_id_luc   VANZARI_PRJ.id_lucrare%TYPE;
```

```
    v_titlu    LUCRARI_ARTA_PRJ.titlu%TYPE;
```

```
BEGIN
```

```
    SELECT num INTO v_num  
    FROM CLIENTI_PRJ  
    WHERE id_client = v_id_client;
```

```
    SELECT COUNT(*), NVL(SUM(pret_final),0)  
    INTO v_nr, v_total  
    FROM VANZARI_PRJ  
    WHERE id_client = v_id_client;
```

```
    DBMS_OUTPUT.PUT_LINE('CLIENT | NR_VANZARI | TOTAL | CATEGORIE');
```

```
    IF v_total >= 8000 THEN
```

```
        DBMS_OUTPUT.PUT_LINE(v_num || ' | ' || v_nr || ' | ' || v_total || ' | VIP');
```

```
    ELSE
```

```
        DBMS_OUTPUT.PUT_LINE(v_num || ' | ' || v_nr || ' | ' || v_total || ' | NORMAL');
```

```
    END IF;
```

```
    IF v_nr > 0 THEN
```

```
        SELECT id_vanzare, pret_final, id_lucrare  
        INTO v_id_v_max, v_p_max, v_id_luc  
        FROM VANZARI_PRJ  
        WHERE id_client = v_id_client  
              AND pret_final = (SELECT MAX(pret_final) FROM VANZARI_PRJ WHERE id_client = v_id_client)  
              FETCH FIRST 1 ROW ONLY;
```

```
        SELECT titlu INTO v_titlu  
        FROM LUCRARI_ARTA_PRJ  
        WHERE id_lucrare = v_id_luc;
```

```
        DBMS_OUTPUT.PUT_LINE('TOP: vanzare ' || v_id_v_max || ' | ' || v_p_max || ' | lucrare:
```

```
    ELSE
```

```
        DBMS_OUTPUT.PUT_LINE('TOP: clientul nu are achizitii.');
```

```
    END IF;
```

```
END;
```

```
/
```

```
CLIENT | NR_VANZARI | TOTAL | CATEGORIE  
Ion Radu | 2 | 7250 | NORMAL  
TOP: vanzare 21 | 3650 | lucrare: Umbre in oras
```

6) Să se creeze un bloc PL/SQL care citește ID_EXPOZITIE și șterge din tabela LUCRARI_EXPOZITII_PRJ toate lucrările care au data_expusa mai veche de un an. După efectuarea operației să se afișeze numărul de înregistrări șterse. cod:

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    v_id_expo LUCRARI_EXPOZITII_PRJ.id_expozitie%TYPE := &id_expozitie;
```

```
    v_sterse  NUMBER;
```

```
BEGIN
```

```
    DELETE FROM LUCRARI_EXPOZITII_PRJ
```

```
    WHERE id_expozitie = v_id_expo
```

```
    AND data_expusa < ADD_MONTHS(SYSDATE, -12);
```

```
    v_sterse := SQL%ROWCOUNT;
```

```
    DBMS_OUTPUT.PUT_LINE('Expozitie ' || v_id_expo || ' | Randuri sterse: ' || v_sterse);
```

```
    COMMIT;
```

```
END;
```

```
/
```



```

SET SERVEROUTPUT ON;

DECLARE
    v_id_expo    LUCRARI_EXPOZITII_PRJ.id_expozitie%TYPE := &id_expozitie;
    v_sterse     NUMBER;
BEGIN
    DELETE FROM LUCRARI_EXPOZITII_PRJ
    WHERE id_expozitie = v_id_expo
        AND data_expusa < ADD_MONTHS(SYSDATE, -12);

    v_sterse := SQL%ROWCOUNT;

    DBMS_OUTPUT.PUT_LINE('Expozitie ' || v_id_expo || ' | Randuri sterse: ' || v_sterse);

    COMMIT;
END;
/

```

Expozitie 1 | Randuri sterse: 5

A doua Parte

Cursori impliciti:

1) Să se creeze un bloc PL/SQL care citește ID_ARTIST de la tastatură și majorează cu 10% prețul estimativ al lucrărilor realizate după anul 2020 de artistul respectiv. La final să se afișeze numărul total de lucrări modificate.cod:

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    v_id_artist LUCRARI_ARTA_PRJ.id_artist%TYPE := &id_artist;
```

```
BEGIN
```

```
    UPDATE LUCRARI_ARTA_PRJ
```

```
    SET pret_estimativ = pret_estimativ * 1.10
```

```
    WHERE id_artist = v_id_artist
```

```
    AND an_realizare > 2020;
```

```
    IF SQL%FOUND THEN
```

```
DBMS_OUTPUT.PUT_LINE('S-au modificat ' || SQL%ROWCOUNT || '  
lucrari.');
```

```
ELSE
```

```
DBMS_OUTPUT.PUT_LINE('Nu exista lucrari care sa respecte conditia.');
```

```
END IF;
```

```
COMMIT;
```

```
END;
```

```
/
```

```
SET SERVEROUTPUT ON;

DECLARE
    v_id_artist LUCRARI_ARTA_PRJ.id_artist%TYPE := &id_artist;
BEGIN
    UPDATE LUCRARI_ARTA_PRJ
    SET pret_estimativ = pret_estimativ * 1.10
    WHERE id_artist = v_id_artist
    AND an_realizare > 2020;

    IF SQL%FOUND THEN
        DBMS_OUTPUT.PUT_LINE('S-au modificat ' || SQL%ROWCOUNT || ' lucrari.');
```

Script Output x

Task completed in 4,041 seconds

```
    ELSE
        DBMS_OUTPUT.PUT_LINE('Nu exista lucrari care sa respecte conditia.');
```

END IF;

COMMIT;

END;

S-au modificat 1 lucrari.

PL/SQL procedure successfully completed.

2) Să se creeze un bloc PL/SQL care citește ID_ARTIST de la tastatură și majorează cu 10% prețul estimativ al lucrărilor realizate de artistul respectiv înainte de anul 2020. La final să se afișeze numărul total de lucrări modificate. cod:

SET SERVEROUTPUT ON;

DECLARE

v_id_artist LUCRARI_ARTA_PRJ.id_artist%TYPE := &id_artist;

BEGIN

UPDATE LUCRARI_ARTA_PRJ

SET pret_estimativ = ROUND(pret_estimativ * 1.10, 2)

WHERE id_artist = v_id_artist

AND an_realizare < 2020;

DBMS_OUTPUT.PUT_LINE('ID_ARTIST | NR_LUCRARI_MODIFICATE');

DBMS_OUTPUT.PUT_LINE(v_id_artist || ' | ' || SQL%ROWCOUNT);

COMMIT;

END;

/

```
SET SERVEROUTPUT ON;

DECLARE
    v_id_artist LUCRARI_ARTA_PRJ.id_artist%TYPE := &id_artist;
BEGIN
    UPDATE LUCRARI_ARTA_PRJ
    SET pret_estimativ = ROUND(pret_estimativ * 1.10, 2)
    WHERE id_artist = v_id_artist
        AND an_realizare < 2020;

    DBMS_OUTPUT.PUT_LINE('ID_ARTIST | NR_LUCRARI_MODIFICATE');
    DBMS_OUTPUT.PUT_LINE(v_id_artist || ' | ' || SQL%ROWCOUNT);

    COMMIT;
END;
/
```

Script Output x

Task completed in 5,58 seconds

```
BEGIN
    UPDATE LUCRARI_ARTA_PRJ
    SET pret_estimativ = ROUND(pret_estimativ * 1.10, 2)
    WHERE id_artist = v_id_artist
        AND an_realizare < 2020;

    DBMS_OUTPUT.PUT_LINE('ID_ARTIST | NR_LUCRARI_MODIFICATE');
    DBMS_OUTPUT.PUT_LINE(v_id_artist || ' | ' || SQL%ROWCOUNT);

    COMMIT;
END;
ID_ARTIST | NR_LUCRARI_MODIFICATE
4 | 1

PL/SQL procedure successfully completed.
```

3) Să se creeze un bloc PL/SQL care citește ID_CLIENT de la tastatură și majorează cu 5% prețul final al tuturor vânzărilor asociate clientului respectiv, pentru care prețul final este mai mic decât 3000. La final să se afișeze numărul total de vânzări modificate. cod:

SET SERVEROUTPUT ON;

DECLARE

v_id_client VANZARI_PRJ.id_client%TYPE := &id_client;

BEGIN

UPDATE VANZARI_PRJ

SET pret_final = ROUND(pret_final * 1.05, 2)

WHERE id_client = v_id_client

AND pret_final < 3000;

DBMS_OUTPUT.PUT_LINE('ID_CLIENT | NR_VANZARI_MODIFICATE');

DBMS_OUTPUT.PUT_LINE(v_id_client || ' | ' || SQL%ROWCOUNT);

COMMIT;

END;

/

```
SET SERVEROUTPUT ON;

DECLARE
    v_id_client VANZARI_PRJ.id_client%TYPE := &id_client;
BEGIN
    UPDATE VANZARI_PRJ
    SET pret_final = ROUND(pret_final * 1.05, 2)
    WHERE id_client = v_id_client
    AND pret_final < 3000;

    DBMS_OUTPUT.PUT_LINE('ID_CLIENT | NR_VANZARI_MODIFICATE');
    DBMS_OUTPUT.PUT_LINE(v_id_client || ' | ' || SQL%ROWCOUNT);

    COMMIT;
END;
/
```

Script Output x

Task completed in 2,61 seconds

```
BEGIN
    UPDATE VANZARI_PRJ
    SET pret_final = ROUND(pret_final * 1.05, 2)
    WHERE id_client = v_id_client
    AND pret_final < 3000;

    DBMS_OUTPUT.PUT_LINE('ID_CLIENT | NR_VANZARI_MODIFICATE');
    DBMS_OUTPUT.PUT_LINE(v_id_client || ' | ' || SQL%ROWCOUNT);

    COMMIT;
END;
ID_CLIENT | NR_VANZARI_MODIFICATE
3 | 0

PL/SQL procedure successfully completed.
```

4) Să se creeze un bloc PL/SQL care citește ID_EXPOZITIE de la tastatură și actualizează locul în expoziție pentru toate lucrările din expoziția respectivă, adăugând prefixul SECTOR- înaintea valorii existente. La final să se afișeze numărul total de înregistrări modificate. cod:

SET SERVEROUTPUT ON;

DECLARE

**v_id_expozitie LUCRARI_EXPOZITII_PRJ.id_expozitie%TYPE :=
&id_expozitie;**

BEGIN

UPDATE LUCRARI_EXPOZITII_PRJ

SET loc_in_expozitie = 'SECTOR-' || loc_in_expozitie

WHERE id_expozitie = v_id_expozitie

AND loc_in_expozitie NOT LIKE 'SECTOR-%';

**DBMS_OUTPUT.PUT_LINE('ID_EXPOZITIE |
NR_INREGISTRARI_MODIFICATE');**

DBMS_OUTPUT.PUT_LINE(v_id_expozitie || ' | ' || SQL%ROWCOUNT);

COMMIT;

END;

/


```
SET SERVEROUTPUT ON;

DECLARE
    v_id_expozitie LUCRARI_EXPOZITII_PRJ.id_expozitie%TYPE := &id_expozitie;
BEGIN
    UPDATE LUCRARI_EXPOZITII_PRJ
    SET loc_in_expozitie = 'SECTOR-' || loc_in_expozitie
    WHERE id_expozitie = v_id_expozitie
    AND loc_in_expozitie NOT LIKE 'SECTOR-%';

    DBMS_OUTPUT.PUT_LINE('ID_EXPOZITIE | NR_INREGISTRARI_MODIFICATE');
    DBMS_OUTPUT.PUT_LINE(v_id_expozitie || ' | ' || SQL%ROWCOUNT);

    COMMIT;
END;
/
```

Script Output x

Task completed in 2,458 seconds

```
BEGIN
    UPDATE LUCRARI_EXPOZITII_PRJ
    SET loc_in_expozitie = 'SECTOR-' || loc_in_expozitie
    WHERE id_expozitie = v_id_expozitie
    AND loc_in_expozitie NOT LIKE 'SECTOR-%';

    DBMS_OUTPUT.PUT_LINE('ID_EXPOZITIE | NR_INREGISTRARI_MODIFICATE');
    DBMS_OUTPUT.PUT_LINE(v_id_expozitie || ' | ' || SQL%ROWCOUNT);

    COMMIT;
END;
ID_EXPOZITIE | NR_INREGISTRARI_MODIFICATE
3 | 5

PL/SQL procedure successfully completed.
```

Cursori expliciti:

5) Să se creeze un bloc PL/SQL care citește ID_EXPOZITIE de la tastatură și afișează toate lucrările expuse în expoziția respectivă, împreună cu titlul lucrării, prețul estimativ și locul în expoziție. La final să se afișeze numărul total de lucrări parcurse. cod:

SET SERVEROUTPUT ON;

DECLARE

v_id_expozitie EXPOZITII_PRJ.id_expozitie%TYPE := &id_expozitie;

CURSOR c_lucrari IS

SELECT l.id_lucrare, l.titlu, l.pret_estimativ, le.loc_in_expozitie

FROM LUCRARI_ARTA_PRJ l, LUCRARI_EXPOZITII_PRJ le

WHERE l.id_lucrare = le.id_lucrare

AND le.id_expozitie = v_id_expozitie

ORDER BY l.id_lucrare;

v_id_lucrare LUCRARI_ARTA_PRJ.id_lucrare%TYPE;

v_titlu LUCRARI_ARTA_PRJ.titlu%TYPE;

v_pret LUCRARI_ARTA_PRJ.pret_estimativ%TYPE;

v_loc LUCRARI_EXPOZITII_PRJ.loc_in_expozitie%TYPE;

BEGIN

OPEN c_lucrari;

**DBMS_OUTPUT.PUT_LINE('ID_LUCRARE | TITLU | PRET_ESTIMATIV |
LOC_IN_EXPOZITIE');**

LOOP

FETCH c_lucrari INTO v_id_lucrare, v_titlu, v_pret, v_loc;

EXIT WHEN c_lucrari%NOTFOUND;

**DBMS_OUTPUT.PUT_LINE(v_id_lucrare || ' | ' || v_titlu || ' | ' || v_pret || ' | ' ||
v_loc);**

END LOOP;

```
DBMS_OUTPUT.PUT_LINE('TOTAL_LUCRARI | ' || c_lucrari%ROWCOUNT);
```

```
CLOSE c_lucrari;
```

```
END;
```

```
/
```

The screenshot displays the SQL Developer interface. The top pane, titled 'Query Builder', contains a PL/SQL script. The script declares four variables: `v_id_lucrare`, `v_titlu`, `v_pret`, and `v_loc`, each associated with a specific table and column. It then begins a block with `BEGIN`, opens a cursor `c_lucrari` with `OPEN c_lucrari;`. A loop is implemented using `LOOP`, `FETCH c_lucrari INTO v_id_lucrare, v_titlu, v_pret, v_loc;`, and `EXIT WHEN c_lucrari%NOTFOUND;`. Inside the loop, the script prints the fetched values using `DBMS_OUTPUT.PUT_LINE(v_id_lucrare || ' | ' || v_titlu || ' | ' || v_pret || ' | ' || v_loc);`. After the loop, it prints the total number of rows with `DBMS_OUTPUT.PUT_LINE('TOTAL_LUCRARI | ' || c_lucrari%ROWCOUNT);`, closes the cursor with `CLOSE c_lucrari;`, and ends the block with `END;`. The bottom pane, titled 'Script Output', shows the execution results. It indicates that the task was completed in 2,258 seconds. The output displays the results of the script execution, including the total number of rows and the fetched data for each row. The output is as follows:

```
DBMS_OUTPUT.PUT_LINE('TOTAL_LUCRARI | ' || c_lucrari%ROWCOUNT);

CLOSE c_lucrari;
END;
ID_LUCRARE | TITLU | PRET_ESTIMATIV | LOC_IN_EXPOZITIE
3 | Strada veche | 3000 | SECTOR-Sala 3 intrare
7 | Portret italian | 3990 | SECTOR-Sala 1 portrete A
10 | Portret de femeie | 3500 | SECTOR-Sala 1 portrete B
11 | Familie in parc | 3300 | SECTOR-Sala 2 perete portrete
23 | Portret batran | 3100 | SECTOR-Sala 2 colt clasic
TOTAL_LUCRARI | 5

PL/SQL procedure successfully completed.
```

6) Să se creeze un bloc PL/SQL care citește ID_ARTIST de la tastatură și afișează lucrările artistului respectiv care au prețul estimativ mai mare decât media prețurilor lucrărilor sale. Pentru fiecare lucrare să se afișeze id-ul, titlul,

anul realizării și prețul estimativ. La final să se afișeze numărul de lucrări găsite. cod:

SET SERVEROUTPUT ON;

DECLARE

v_id_artist ARTISTI_PRJ.id_artist%TYPE := &id_artist;

CURSOR c_lucrari IS

SELECT id_lucrare, titlu, an_realizare, pret_estimativ

FROM LUCRARI_ARTA_PRJ

WHERE id_artist = v_id_artist

AND pret_estimativ > (

SELECT AVG(pret_estimativ)

FROM LUCRARI_ARTA_PRJ

WHERE id_artist = v_id_artist

)

ORDER BY pret_estimativ DESC;

v_id_lucrare LUCRARI_ARTA_PRJ.id_lucrare%TYPE;

v_titlu LUCRARI_ARTA_PRJ.titlu%TYPE;

v_an LUCRARI_ARTA_PRJ.an_realizare%TYPE;

v_pret LUCRARI_ARTA_PRJ.pret_estimativ%TYPE;

BEGIN

OPEN c_lucrari;

```
DBMS_OUTPUT.PUT_LINE('ID_LUCRARE | TITLU | AN_REALIZARE |  
PRET_ESTIMATIV');
```

```
LOOP
```

```
FETCH c_lucrari INTO v_id_lucrare, v_titlu, v_an, v_pret;
```

```
EXIT WHEN c_lucrari%NOTFOUND;
```

```
DBMS_OUTPUT.PUT_LINE(v_id_lucrare || ' | ' || v_titlu || ' | ' || v_an || ' | ' ||  
v_pret);
```

```
END LOOP;
```

```
DBMS_OUTPUT.PUT_LINE('TOTAL_LUCRARI | ' || c_lucrari%ROWCOUNT);
```

```
CLOSE c_lucrari;
```

```
END;
```

```
/
```

Worksheet
Query Builder

```

v_id_lucrare LUCRARI_ARTA_PRJ.id_lucrare%TYPE;
v_titlu LUCRARI_ARTA_PRJ.titlu%TYPE;
v_an LUCRARI_ARTA_PRJ.an_realizare%TYPE;
v_pret LUCRARI_ARTA_PRJ.pret_estimativ%TYPE;

BEGIN
  OPEN c_lucrari;

  DBMS_OUTPUT.PUT_LINE('ID_LUCRARE | TITLU | AN_REALIZARE | PRET_ESTIMATIV');

  LOOP
    FETCH c_lucrari INTO v_id_lucrare, v_titlu, v_an, v_pret;
    EXIT WHEN c_lucrari%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE(v_id_lucrare || ' | ' || v_titlu || ' | ' || v_an || ' | ' || v_pret);
  END LOOP;

  DBMS_OUTPUT.PUT_LINE('TOTAL_LUCRARI | ' || c_lucrari%ROWCOUNT);

  CLOSE c_lucrari;
END;
/

```

Script Output x
Task completed in 2,843 seconds

```

EXIT WHEN c_lucrari%NOTFOUND;

DBMS_OUTPUT.PUT_LINE(v_id_lucrare || ' | ' || v_titlu || ' | ' || v_an || ' | ' || v_pret);
END LOOP;

DBMS_OUTPUT.PUT_LINE('TOTAL_LUCRARI | ' || c_lucrari%ROWCOUNT);

CLOSE c_lucrari;
END;
ID_LUCRARE | TITLU | AN_REALIZARE | PRET_ESTIMATIV
5 | Forme in miscare | 2021 | 3520
TOTAL_LUCRARI | 1

```

PL/SQL procedure successfully completed.

7) Să se creeze un bloc PL/SQL care citește ID_CLIENT de la tastatură și afișează toate vânzările realizate de clientul respectiv, împreună cu titlul lucrării, data vânzării și prețul final. La final să se determine valoarea totală a cumpărăturilor și categoria clientului: BRONZE dacă totalul este sub 5000, SILVER dacă totalul este între 5000 și 10000, respectiv GOLD dacă totalul depășește 10000.cod:

SET SERVEROUTPUT ON;

DECLARE

v_id_client CLIENTI_PRJ.id_client%TYPE := &id_client;

CURSOR c_vanzari IS

SELECT v.id_vanzare, l.titlu, v.data_vanzare, v.pret_final

FROM VANZARI_PRJ v, LUCRARI_ARTA_PRJ l

WHERE v.id_lucrare = l.id_lucrare

AND v.id_client = v_id_client

ORDER BY v.data_vanzare;

v_id_vanzare VANZARI_PRJ.id_vanzare%TYPE;

v_titlu LUCRARI_ARTA_PRJ.titlu%TYPE;

v_data VANZARI_PRJ.data_vanzare%TYPE;

v_pret VANZARI_PRJ.pret_final%TYPE;

v_total NUMBER := 0;

BEGIN

OPEN c_vanzari;

**DBMS_OUTPUT.PUT_LINE('ID_VANZARE | TITLU | DATA_VANZARE |
PRET_FINAL');**

LOOP

FETCH c_vanzari INTO v_id_vanzare, v_titlu, v_data, v_pret;

EXIT WHEN c_vanzari%NOTFOUND;

DBMS_OUTPUT.PUT_LINE(v_id_vanzare || ' | ' || v_titlu || ' | ' ||

TO_CHAR(v_data,'DD-MM-YYYY') || ' | ' || v_pret);

```
        v_total := v_total + v_pret;

END LOOP;


CLOSE c_vanzari;


DBMS_OUTPUT.PUT_LINE('TOTAL_CUMPARATURI | ' || v_total);


IF v_total < 5000 THEN

    DBMS_OUTPUT.PUT_LINE('CATEGORIE | BRONZE');

ELSIF v_total <= 10000 THEN

    DBMS_OUTPUT.PUT_LINE('CATEGORIE | SILVER');

ELSE

    DBMS_OUTPUT.PUT_LINE('CATEGORIE | GOLD');

END IF;

END;

/
```



```
LOOP
    FETCH c_vanzari INTO v_id_vanzare, v_titlu, v_data, v_pret;
    EXIT WHEN c_vanzari%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE(v_id_vanzare || ' | ' || v_titlu || ' | ' ||
        TO_CHAR(v_data, 'DD-MM-YYYY') || ' | ' || v_pret);

    v_total := v_total + v_pret;
END LOOP;

CLOSE c_vanzari;

DBMS_OUTPUT.PUT_LINE('TOTAL_CUMPARATURI | ' || v_total);

IF v_total < 5000 THEN
    DBMS_OUTPUT.PUT_LINE('CATEGORIE | BRONZE');
ELSIF v_total <= 10000 THEN
    DBMS_OUTPUT.PUT_LINE('CATEGORIE | SILVER');
ELSE
    DBMS_OUTPUT.PUT_LINE('CATEGORIE | GOLD');
END IF;
END;
/
```

Script Output x

Task completed in 2,964 seconds

```
DBMS_OUTPUT.PUT_LINE('CATEGORIE | BRONZE');
ELSIF v_total <= 10000 THEN
    DBMS_OUTPUT.PUT_LINE('CATEGORIE | SILVER');
ELSE
    DBMS_OUTPUT.PUT_LINE('CATEGORIE | GOLD');
END IF;
END;
ID_VANZARE | TITLU | DATA_VANZARE | PRET_FINAL
2 | Blocuri de lumina | 20-03-2022 | 2850
22 | Texturi urbane | 28-08-2022 | 3450
TOTAL_CUMPARATURI | 6300
CATEGORIE | SILVER

PL/SQL procedure successfully completed.
```

8) Să se creeze un bloc PL/SQL care afișează pentru fiecare artist informații despre lucrările sale. Pentru fiecare artist să se afișeze id-ul și numele, iar pentru fiecare lucrare id-ul, titlul și prețul estimativ. La nivelul fiecărui artist să se afișeze și valoarea totală estimativă a lucrărilor sale.cod:

SET SERVEROUTPUT ON;

DECLARE

CURSOR c_artisti IS

SELECT id_artist, nume

FROM ARTISTI_PRJ

ORDER BY id_artist;

CURSOR c_lucrari(p_artist NUMBER) IS

SELECT id_lucrare, titlu, pret_estimativ

FROM LUCRARI_ARTA_PRJ

WHERE id_artist = p_artist

ORDER BY id_lucrare;

v_id_artist ARTISTI_PRJ.id_artist%TYPE;

v_nume_artist ARTISTI_PRJ.numa_artist%TYPE;

v_id_lucrare LUCRARI_ARTA_PRJ.id_lucrare%TYPE;

v_titlu LUCRARI_ARTA_PRJ.titlu%TYPE;

v_pret LUCRARI_ARTA_PRJ.pret_estimativ%TYPE;

v_total NUMBER;

BEGIN

OPEN c_artisti;

LOOP

FETCH c_artisti INTO v_id_artist, v_nume_artist;

EXIT WHEN c_artisti%NOTFOUND;

DBMS_OUTPUT.PUT_LINE('ARTIST | ' || v_id_artist || ' | ' || v_nume_artist);

DBMS_OUTPUT.PUT_LINE('ID_LUCRARE | TITLU | PRET_ESTIMATIV');

v_total := 0;

OPEN c_lucrari(v_id_artist);

LOOP

FETCH c_lucrari INTO v_id_lucrare, v_titlu, v_pret;

EXIT WHEN c_lucrari%NOTFOUND;

DBMS_OUTPUT.PUT_LINE(v_id_lucrare || ' | ' || v_titlu || ' | ' || v_pret);

v_total := v_total + v_pret;

END LOOP;

CLOSE c_lucrari;

DBMS_OUTPUT.PUT_LINE('TOTAL_ARTIST | ' || v_total);

DBMS_OUTPUT.PUT_LINE('-----');

END LOOP;

CLOSE c_artisti;

END;

/

ARTIST | 1 | Andrei Popescu
ID_LUCRARE | TITLU | PRET_ESTIMATIV
1 | Noapte in Bucuresti | 3500
2 | Blocuri de lumina | 3388
3 | Strada veche | 3000
TOTAL_ARTIST | 9888

ARTIST | 2 | Maria Ionescu
ID_LUCRARE | TITLU | PRET_ESTIMATIV
4 | Abstractie albastra | 2860
5 | Forme in miscare | 3520
TOTAL_ARTIST | 6380

ARTIST | 3 | Luca Bianchi
ID_LUCRARE | TITLU | PRET_ESTIMATIV
6 | Piazza veche | 4000
7 | Portret italian | 3990
23 | Portret batran | 3100
TOTAL_ARTIST | 11090

ARTIST | 4 | Sophie Dubois
ID_LUCRARE | TITLU | PRET_ESTIMATIV
8 | Linii de Paris | 2100
9 | Minimal Paris | 2090
24 | Podul peste Sena | 2420
TOTAL_ARTIST | 6610

ARTIST | 5 | Emily Johnson
ID_LUCRARE | TITLU | PRET_ESTIMATIV
10 | Portret de femeie | 3500
11 | Familie in parc | 3300
TOTAL_ARTIST | 6800

ARTIST | 6 | Kenji Tanaka
ID_LUCRARE | TITLU | PRET_ESTIMATIV
12 | Tokyo Night | 2750
13 | Fuziune traditionala | 3800
TOTAL_ARTIST | 6550

ARTIST | 7 | Carlos Martinez

```

-----
ARTIST | 6 | Kenji Tanaka
ID_LUCRARE | TITLU | PRET_ESTIMATIV
12 | Tokyo Night | 2750
13 | Fuziune traditionala | 3800
TOTAL_ARTIST | 6550
-----
ARTIST | 7 | Carlos Martinez
ID_LUCRARE | TITLU | PRET_ESTIMATIV
14 | Soare iberic | 2700
15 | Dansul culorilor | 3190
TOTAL_ARTIST | 5890
-----
ARTIST | 8 | Anna Muller
ID_LUCRARE | TITLU | PRET_ESTIMATIV
16 | Forme geometrice I | 2000
17 | Forme geometrice II | 2100
TOTAL_ARTIST | 4100
-----
ARTIST | 9 | Elena Petrescu
ID_LUCRARE | TITLU | PRET_ESTIMATIV
18 | Lacul la apus | 1800
19 | Copaci in ceata | 2090
TOTAL_ARTIST | 3890
-----
ARTIST | 10 | Oliver Smith
ID_LUCRARE | TITLU | PRET_ESTIMATIV
20 | Instalatie luminoasa | 4987,5
21 | Umbre in oras | 3780
22 | Texturi urbane | 3570
TOTAL_ARTIST | 12337,5
-----

PL/SQL procedure successfully completed.

```

9) Să se creeze un bloc PL/SQL care afișează pentru fiecare expoziție informații despre lucrările expuse și prețul estimativ al acestora. Pentru fiecare expoziție să se afișeze id-ul și denumirea, iar pentru fiecare lucrare expusă id-ul, titlul și prețul estimativ. La nivelul fiecărei expoziții să se afișeze și valoarea totală estimativă a lucrărilor expuse.cod:

SET SERVEROUTPUT ON;

DECLARE

CURSOR c_expozitii IS

SELECT id_expozitie, nume

FROM EXPOZITII_PRJ

ORDER BY id_expozitie;

```

CURSOR c_lucrari(p_expozitie NUMBER) IS

    SELECT l.id_lucrare, l.titlu, l.pret_estimativ

    FROM LUCRARI_ARTA_PRJ l, LUCRARI_EXPOZITII_PRJ le

    WHERE l.id_lucrare = le.id_lucrare

        AND le.id_expozitie = p_expozitie

    ORDER BY l.id_lucrare;


v_id_expozitie EXPOZITII_PRJ.id_expozitie%TYPE;

v_nume_expozitie EXPOZITII_PRJ.nume%TYPE;


v_id_lucrare LUCRARI_ARTA_PRJ.id_lucrare%TYPE;

v_titlu LUCRARI_ARTA_PRJ.titlu%TYPE;

v_pret LUCRARI_ARTA_PRJ.pret_estimativ%TYPE;


v_total NUMBER;

BEGIN

    OPEN c_expozitii;


    LOOP

        FETCH c_expozitii INTO v_id_expozitie, v_nume_expozitie;

        EXIT WHEN c_expozitii%NOTFOUND;


        DBMS_OUTPUT.PUT_LINE('EXPOZITIE | ' || v_id_expozitie || ' | ' ||
v_nume_expozitie);

        DBMS_OUTPUT.PUT_LINE('ID_LUCRARE | TITLU | PRET_ESTIMATIV');

```

v_total := 0;

OPEN c_lucrari(v_id_expozitie);

LOOP

FETCH c_lucrari INTO v_id_lucrare, v_titlu, v_pret;

EXIT WHEN c_lucrari%NOTFOUND;

DBMS_OUTPUT.PUT_LINE(v_id_lucrare || ' | ' || v_titlu || ' | ' || v_pret);

v_total := v_total + v_pret;

END LOOP;

CLOSE c_lucrari;

DBMS_OUTPUT.PUT_LINE('TOTAL_EXPOZITIE | ' || v_total);

DBMS_OUTPUT.PUT_LINE('-----');

END LOOP;

CLOSE c_expozitii;

END;

/

EXPOZITIE | 1 | Culori Urbane
ID_LUCRARE | TITLU | PRET_ESTIMATIV
4 | Abstractie albastra | 2860
5 | Forme in miscare | 3520
6 | Piazza veche | 4000
7 | Portret italian | 3990
8 | Linii de Paris | 2100
9 | Minimal Paris | 2090
10 | Portret de femeie | 3500
TOTAL_EXPOZITIE | 22060

EXPOZITIE | 2 | Abstract si Forme
ID_LUCRARE | TITLU | PRET_ESTIMATIV
4 | Abstractie albastra | 2860
5 | Forme in miscare | 3520
12 | Tokyo Night | 2750
13 | Fuziune traditionala | 3800
16 | Forme geometrice I | 2000
17 | Forme geometrice II | 2100
TOTAL_EXPOZITIE | 17030

EXPOZITIE | 3 | Portrete Contemporane
ID_LUCRARE | TITLU | PRET_ESTIMATIV
3 | Strada veche | 3000
7 | Portret italian | 3990
10 | Portret de femeie | 3500
11 | Familie in parc | 3300
23 | Portret batran | 3100
TOTAL_EXPOZITIE | 16890

EXPOZITIE | 4 | Lumi Imaginare
ID_LUCRARE | TITLU | PRET_ESTIMATIV
8 | Linii de Paris | 2100
9 | Minimal Paris | 2090
14 | Soare iberic | 2700
15 | Dansul culorilor | 3190
20 | Instalatie luminoasa | 4987,5
TOTAL_EXPOZITIE | 15067,5

EXPOZITIE | 5 | Minimalism si Linie
ID_LUCRARE | TITLU | PRET_ESTIMATIV
8 | Linii de Paris | 2100
9 | Minimal Paris | 2090
16 | Forme geometrice I | 2000
17 | Forme geometrice II | 2100
24 | Podul peste Sena | 2420
TOTAL_EXPOZITIE | 10710

EXPOZITIE | 6 | Peisaje si Natura
ID_LUCRARE | TITLU | PRET_ESTIMATIV
1 | Noapte in Bucuresti | 3500
2 | Blocuri de lumina | 3388
6 | Piazza veche | 4000
18 | Lacul la apus | 1800
19 | Copaci in ceata | 2090
TOTAL_EXPOZITIE | 14778

PL/SQL procedure successfully completed.

Partea a 3 – a

1) Să se creeze un bloc PL/SQL care citește ID_CLIENT de la tastatură și afișează numele, emailul și telefonul clientului respectiv. Dacă nu există clientul cu id-ul introdus, să se trateze excepția corespunzătoare și să se afișeze un mesaj. Cod: SET SERVEROUTPUT ON;

DECLARE

v_id_client CLIENTI_PRJ.id_client%TYPE := &id_client;

v_nume CLIENTI_PRJ.numename%TYPE;

v_email CLIENTI_PRJ.email%TYPE;

v_telefon CLIENTI_PRJ.telefon%TYPE;

BEGIN

SELECT nume, email, telefon

INTO v_nume, v_email, v_telefon

FROM CLIENTI_PRJ

WHERE id_client = v_id_client;

DBMS_OUTPUT.PUT_LINE('ID_CLIENT | NUME | EMAIL | TELEFON');

DBMS_OUTPUT.PUT_LINE(v_id_client || ' | ' || v_nume || ' | ' || v_email || ' | ' || v_telefon);

EXCEPTION

WHEN NO_DATA_FOUND THEN

DBMS_OUTPUT.PUT_LINE('Nu exista client cu acest ID.');

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('A aparut o alta exceptie.');

END;

/

ID_CLIENT	NUME	EMAIL	TELEFON
3	Ana Marinescu	ana.m@galerie.ro	+40 723333333

PL/SQL procedure successfully completed.

2) Să se creeze un bloc PL/SQL care citește de la tastatură o țară de origine și încearcă să afișeze numele unui artist din țara respectivă. Dacă nu există niciun artist din țara respectivă, să se trateze excepția NO_DATA_FOUND. Dacă există mai mulți artiști, să se trateze excepția TOO_MANY_ROWS. Cod:

SET SERVEROUTPUT ON;

DECLARE

v_tara ARTISTI_PRJ.tara_origine%TYPE := '&tara_origine';

v_nume ARTISTI_PRJ.nume%TYPE;

BEGIN

SELECT nume

INTO v_nume

FROM ARTISTI_PRJ

WHERE UPPER(tara_origine) = UPPER(v_tara);

DBMS_OUTPUT.PUT_LINE('Artist: ' || v_nume);

EXCEPTION

WHEN NO_DATA_FOUND THEN

DBMS_OUTPUT.PUT_LINE('Nu exista artisti din tara introdusa.');

WHEN TOO_MANY_ROWS THEN

DBMS_OUTPUT.PUT_LINE('Exista mai multi artisti din tara introdusa.');

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('A aparut o alta exceptie.');

END;

/

Exista mai multi artisti din tara introdusa.

PL/SQL procedure successfully completed.

3) Să se creeze un bloc PL/SQL care citește ID_CLIENT și afișează numele clientului și valoarea totală a cumpărăturilor sale. Dacă clientul există, dar nu are nicio vânzare înregistrată, să se invoce o excepție definită de utilizator și să se trateze corespunzător. Cod:

SET SERVEROUTPUT ON;

DECLARE

v_id_client CLIENTI_PRJ.id_client%TYPE := &id_client;

v_nume CLIENTI_PRJ.nume%TYPE;

v_total NUMBER;

ex_fara_vanzari EXCEPTION;

BEGIN

SELECT nume

INTO v_nume

FROM CLIENTI_PRJ

WHERE id_client = v_id_client;

SELECT NVL(SUM(pret_final), 0)

INTO v_total

FROM VANZARI_PRJ

WHERE id_client = v_id_client;

IF v_total = 0 THEN

RAISE ex_fara_vanzari;

END IF;

DBMS_OUTPUT.PUT_LINE('CLIENT | TOTAL_CUMPARATURI');

DBMS_OUTPUT.PUT_LINE(v_nume || ' | ' || v_total);

EXCEPTION

WHEN NO_DATA_FOUND THEN

DBMS_OUTPUT.PUT_LINE('Clientul nu exista.');

WHEN ex_fara_vanzari THEN

DBMS_OUTPUT.PUT_LINE('Clientul exista, dar nu are vanzari inregistrate.');

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('A aparut o alta exceptie.');

END;

/

```
CLIENT | TOTAL_CUMPARATURI  
Mihai Georgescu | 6300
```

PL/SQL procedure successfully completed.

4) Să se creeze un bloc PL/SQL care citește ID_EXPOZITIE și afișează toate lucrările expuse în expoziția respectivă, împreună cu titlul și locul în expoziție. Dacă expoziția există, dar nu are lucrări expuse, să se invoce o excepție definită de utilizator și să se trateze corespunzător. Cod:

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    v_id_expozitie EXPOZITII_PRJ.id_expozitie%TYPE := &id_expozitie;
```

```
    v_nume_expozitie EXPOZITII_PRJ.numename%TYPE;
```

```
CURSOR c_lucrari IS
```

```
    SELECT l.id_lucrare, l.titlu, le.loc_in_expozitie
```

```
    FROM LUCRARI_ARTA_PRJ l, LUCRARI_EXPOZITII_PRJ le
```

```
    WHERE l.id_lucrare = le.id_lucrare
```

```
    AND le.id_expozitie = v_id_expozitie
```

```
    ORDER BY l.id_lucrare;
```

```
    v_id_lucrare LUCRARI_ARTA_PRJ.id_lucrare%TYPE;
```

```
    v_titlu LUCRARI_ARTA_PRJ.titlu%TYPE;
```

```
    v_loc LUCRARI_EXPOZITII_PRJ.loc_in_expozitie%TYPE;
```

```
    v_nr NUMBER := 0;
```

```
    ex_fara_lucrari EXCEPTION;
```

```
BEGIN
```

```
    SELECT nume
```

```
    INTO v_nume_expozitie
```

```
    FROM EXPOZITII_PRJ
```

WHERE id_expozitie = v_id_expozitie;

OPEN c_lucrari;

DBMS_OUTPUT.PUT_LINE('EXPOZITIE | ' || v_nume_expozitie);

DBMS_OUTPUT.PUT_LINE('ID_LUCRARE | TITLU | LOC');

LOOP

FETCH c_lucrari INTO v_id_lucrare, v_titlu, v_loc;

EXIT WHEN c_lucrari%NOTFOUND;

DBMS_OUTPUT.PUT_LINE(v_id_lucrare || ' | ' || v_titlu || ' | ' || v_loc);

v_nr := v_nr + 1;

END LOOP;

CLOSE c_lucrari;

IF v_nr = 0 THEN

RAISE ex_fara_lucrari;

END IF;

DBMS_OUTPUT.PUT_LINE('TOTAL_LUCRARI | ' || v_nr);

EXCEPTION

WHEN NO_DATA_FOUND THEN

```

        DBMS_OUTPUT.PUT_LINE('Expozitia nu exista.');
```

WHEN ex_fara_lucrari THEN

```

        DBMS_OUTPUT.PUT_LINE('Expozitia exista, dar nu are lucrari expuse.');
```

WHEN OTHERS THEN

```

        DBMS_OUTPUT.PUT_LINE('A aparut o alta exceptie.');
```

END;

/

```

EXPOZITIE | Portrete Contemporane
ID_LUCRARE | TITLU | LOC
3 | Strada veche | SECTOR-Sala 3 intrare
7 | Portret italian | SECTOR-Sala 1 portrete A
10 | Portret de femeie | SECTOR-Sala 1 portrete B
11 | Familie in parc | SECTOR-Sala 2 perete portret
23 | Portret batran | SECTOR-Sala 2 colt clasic
TOTAL_LUCRARI | 5

```

PL/SQL procedure successfully completed.

5) Să se creeze un bloc PL/SQL care citește ID_ARTIST, afișează numele artistului și calculează valoarea totală estimativă a lucrărilor sale. Dacă artistul există, dar nu are nicio lucrare înregistrată, să se invoce o excepție definită de utilizator și să se trateze corespunzător. În funcție de valoarea totală, artistul va fi clasificat ca MIC, MEDIU sau IMPORTANT. Cod:

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    v_id_artist ARTISTI_PRJ.id_artist%TYPE := &id_artist;
```

```
    v_nume      ARTISTI_PRJ.numename%TYPE;
```

```
    v_total     NUMBER;
```

```
    ex_fara_lucrari EXCEPTION;
```

```
BEGIN
```

```
SELECT nume  
INTO v_nume  
FROM ARTISTI_PRJ  
WHERE id_artist = v_id_artist;
```

```
SELECT NVL(SUM(pret_estimativ), 0)  
INTO v_total  
FROM LUCRARI_ARTA_PRJ  
WHERE id_artist = v_id_artist;
```

```
IF v_total = 0 THEN  
    RAISE ex_fara_lucrari;  
END IF;
```

```
DBMS_OUTPUT.PUT_LINE('ARTIST | TOTAL_ESTIMATIV | CATEGORIE');
```

```
IF v_total < 5000 THEN  
    DBMS_OUTPUT.PUT_LINE(v_nume || ' | ' || v_total || ' | MIC');  
ELSIF v_total <= 15000 THEN  
    DBMS_OUTPUT.PUT_LINE(v_nume || ' | ' || v_total || ' | MEDIU');  
ELSE  
    DBMS_OUTPUT.PUT_LINE(v_nume || ' | ' || v_total || ' | IMPORTANT');  
END IF;
```

```
EXCEPTION
```


WHEN NO_DATA_FOUND THEN

DBMS_OUTPUT.PUT_LINE('Artistul nu exista.');

WHEN ex_fara_lucrari THEN

DBMS_OUTPUT.PUT_LINE('Artistul exista, dar nu are lucrari inregistrate.');

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('A aparut o alta exceptie.');

END;

/

ARTIST	TOTAL_ESTIMATIV	CATEGORIE
Andrei Popescu	9888	MEDIU

PL/SQL procedure successfully completed.

6) Să se creeze un bloc PL/SQL care citește ID_CLIENT și afișează toate vânzările realizate de clientul respectiv, împreună cu titlul lucrării și prețul final. Dacă clientul există, dar nu are achiziții, să se invoce o excepție definită de utilizator și să se trateze corespunzător. Cod:

SET SERVEROUTPUT ON;

DECLARE

v_id_client CLIENTI_PRJ.id_client%TYPE := &id_client;

v_nume CLIENTI_PRJ.numename%TYPE;

CURSOR c_vanzari(p_client NUMBER) IS

SELECT v.id_vanzare, l.titlu, v.pret_final

FROM VANZARI_PRJ v, LUCRARI_ARTA_PRJ l

WHERE v.id_lucrare = l.id_lucrare

AND v.id_client = p_client

ORDER BY v.id_vanzare;

v_id_vanzare VANZARI_PRJ.id_vanzare%TYPE;

v_titlu LUCRARI_ARTA_PRJ.titlu%TYPE;

v_pret VANZARI_PRJ.pret_final%TYPE;

v_nr NUMBER := 0;

ex_fara_achizitii EXCEPTION;

BEGIN

SELECT nume

INTO v_nume

FROM CLIENTI_PRJ

WHERE id_client = v_id_client;

OPEN c_vanzari(v_id_client);

DBMS_OUTPUT.PUT_LINE('CLIENT | ' || v_nume);

DBMS_OUTPUT.PUT_LINE('ID_VANZARE | TITLU | PRET_FINAL');

LOOP

FETCH c_vanzari INTO v_id_vanzare, v_titlu, v_pret;

EXIT WHEN c_vanzari%NOTFOUND;

DBMS_OUTPUT.PUT_LINE(v_id_vanzare || ' | ' || v_titlu || ' | ' || v_pret);

v_nr := v_nr + 1;

END LOOP;

CLOSE c_vanzari;

IF v_nr = 0 THEN

RAISE ex_fara_achizitii;

END IF;

DBMS_OUTPUT.PUT_LINE('TOTAL_VANZARI | ' || v_nr);

EXCEPTION

WHEN NO_DATA_FOUND THEN

DBMS_OUTPUT.PUT_LINE('Clientul nu exista.');

WHEN ex_fara_achizitii THEN

DBMS_OUTPUT.PUT_LINE('Clientul exista, dar nu are achizitii.');

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('A aparut o alta exceptie.');

END;

/

```
CLIENT | George Iancu
ID_VANZARE | TITLU | PRET_FINAL
5 | Forme in miscare | 3300
25 | Noapte in Bucuresti | 3700
TOTAL_VANZARI | 2
```

PL/SQL procedure successfully completed.

7) Să se creeze un bloc PL/SQL care citește ID_ARTIST și afișează lucrările artistului respectiv care au prețul estimativ mai mare decât media prețurilor lucrărilor sale. Dacă artistul nu există, să se trateze NO_DATA_FOUND. Dacă artistul există, dar nu are nicio lucrare peste medie, să se invoce o excepție definită de utilizator și să se trateze corespunzător. Cod:

SET SERVEROUTPUT ON;

DECLARE

v_id_artist ARTISTI_PRJ.id_artist%TYPE := &id_artist;

v_nume ARTISTI_PRJ.nume%TYPE;

CURSOR c_lucrari IS

SELECT id_lucrare, titlu, pret_estimativ

FROM LUCRARI_ARTA_PRJ

WHERE id_artist = v_id_artist

AND pret_estimativ > (

SELECT AVG(pret_estimativ)

FROM LUCRARI_ARTA_PRJ

WHERE id_artist = v_id_artist

)

ORDER BY pret_estimativ DESC;

v_id_lucrare LUCRARI_ARTA_PRJ.id_lucrare%TYPE;

v_titlu LUCRARI_ARTA_PRJ.titlu%TYPE;

v_pret LUCRARI_ARTA_PRJ.pret_estimativ%TYPE;

v_nr NUMBER := 0;

ex_fara_lucrari EXCEPTION;

BEGIN

SELECT nume

INTO v_nume

FROM ARTISTI_PRJ

WHERE id_artist = v_id_artist;

OPEN c_lucrari;

DBMS_OUTPUT.PUT_LINE('ARTIST | ' || v_nume);

DBMS_OUTPUT.PUT_LINE('ID_LUCRARE | TITLU | PRET_ESTIMATIV');

LOOP

FETCH c_lucrari INTO v_id_lucrare, v_titlu, v_pret;

EXIT WHEN c_lucrari%NOTFOUND;

DBMS_OUTPUT.PUT_LINE(v_id_lucrare || ' | ' || v_titlu || ' | ' || v_pret);

v_nr := v_nr + 1;

END LOOP;

CLOSE c_lucrari;

IF v_nr = 0 THEN

RAISE ex_fara_lucrari;

END IF;

```
DBMS_OUTPUT.PUT_LINE('TOTAL_LUCRARI | ' || v_nr);
```

EXCEPTION

WHEN NO_DATA_FOUND THEN

DBMS_OUTPUT.PUT_LINE('Artistul nu exista.');

WHEN ex_fara_lucrari THEN

DBMS_OUTPUT.PUT_LINE('Artistul exista, dar nu are lucrari peste media sa.');

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('A aparut o alta exceptie.');

END;

/

```
ARTIST | Kenji Tanaka  
ID_LUCRARE | TITLU | PRET_ESTIMATIV  
13 | Fuziune traditionala | 3800  
TOTAL_LUCRARI | 1
```

PL/SQL procedure successfully completed.

Partea a 4 – a

1) Să se construiască funcția *fisa_artist*, care primește ca parametru *ID_ARTIST* și returnează un șir de caractere ce conține numele artistului, țara de origine, numărul de lucrări realizate și valoarea totală estimativă a acestora. Să se trateze cazul în care artistul nu există. Cod:

CREATE OR REPLACE FUNCTION *fisa_artist* (*p_id* IN *ARTISTI_PRJ.id_artist%*TYPE)

RETURN VARCHAR2

AS

v_nume ARTISTI_PRJ.numename%TYPE;

v_tara ARTISTI_PRJ.tara_origine%TYPE;

v_nr NUMBER;

v_total NUMBER;

BEGIN

SELECT nume, tara_origine

INTO v_nume, v_tara

FROM ARTISTI_PRJ

WHERE id_artist = p_id;

SELECT COUNT(*), NVL(SUM(pret_estimativ), 0)

INTO v_nr, v_total

FROM LUCRARI_ARTA_PRJ

WHERE id_artist = p_id;

**RETURN v_nume || ' | ' || v_tara || ' | lucrari: ' || v_nr || ' | total estimativ: ' ||
v_total;**

EXCEPTION

WHEN NO_DATA_FOUND THEN

RETURN 'Nu exista artistul cu id-ul ' || p_id;

END;

/

--apel

SELECT fisa_artist(1) FROM dual;

SELECT fisa_artist(100) FROM dual;

```
CREATE OR REPLACE FUNCTION fisa_artist (p_id IN ARTISTI_PRJ.id_artist%TYPE)
RETURN VARCHAR2
AS
    v_nume    ARTISTI_PRJ.numename%TYPE;
    v_tara    ARTISTI_PRJ.tara_origine%TYPE;
    v_nr      NUMBER;
    v_total   NUMBER;
BEGIN
    SELECT nume, tara_origine
    INTO v_nume, v_tara
    FROM ARTISTI_PRJ
    WHERE id_artist = p_id;

    SELECT COUNT(*), NVL(SUM(pret_estimativ), 0)
    INTO v_nr, v_total
    FROM LUCRARI_ARTA_PRJ
    WHERE id_artist = p_id;

    RETURN v_nume || ' ' || v_tara || ' | lucrari: ' || v_nr || ' | total estimati
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 'Nu exista artistul cu id-ul ' || p_id;
END;
/
--apel
SELECT fisa_artist(1) FROM dual;
SELECT fisa_artist(100) FROM dual;
```

Script Output x Query Result x Query Result 1 x

SQL | All Rows Fetched: 1 in 0,12 seconds

FISA_ARTIST(100)
1 Nu exista artistul cu id-ul 100


```

CREATE OR REPLACE FUNCTION fisa_artist (p_id IN ARTISTI_PRJ.id_artist%TYPE)
RETURN VARCHAR2
AS
    v_nume    ARTISTI_PRJ.numename%TYPE;
    v_tara    ARTISTI_PRJ.tara_origine%TYPE;
    v_nr      NUMBER;
    v_total   NUMBER;
BEGIN
    SELECT nume, tara_origine
    INTO v_nume, v_tara
    FROM ARTISTI_PRJ
    WHERE id_artist = p_id;

    SELECT COUNT(*), NVL(SUM(pret_estimativ), 0)
    INTO v_nr, v_total
    FROM LUCRARI_ARTA_PRJ
    WHERE id_artist = p_id;

    RETURN v_nume || ' | ' || v_tara || ' | lucrari: ' || v_nr || ' | total estimati
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 'Nu exista artistul cu id-ul ' || p_id;
END;
/
--apel
SELECT fisa_artist(1) FROM dual;
SELECT fisa_artist(100) FROM dual;

```

Script Output x Query Result x Query Result 1 x

SQL | All Rows Fetched: 1 in 0,189 seconds

FISA_ARTIST(1)			
1	Andrei Popescu	Romania	lucrari: 3 total estimativ: 9888

2) Să se construiască funcția *categorie_client*, care primește ca parametru *ID_CLIENT* și returnează categoria clientului în funcție de suma totală a achizițiilor: *NORMAL*, *VIP* sau *PREMIUM*. Să se trateze cazul în care clientul nu există. Cod:

```

CREATE OR REPLACE FUNCTION categorie_client (p_id IN
CLIENTI_PRJ.id_client%TYPE)

```

```

RETURN VARCHAR2

```

```

AS

```

```

    v_exista NUMBER;

```

```

    v_total NUMBER;

```

```
BEGIN

    SELECT COUNT(*)
    INTO v_exista
    FROM CLIENTI_PRJ
    WHERE id_client = p_id;

    IF v_exista = 0 THEN
        RETURN 'CLIENT INEXISTENT';
    END IF;

    SELECT NVL(SUM(pret_final), 0)
    INTO v_total
    FROM VANZARI_PRJ
    WHERE id_client = p_id;

    IF v_total < 5000 THEN
        RETURN 'NORMAL';
    ELSIF v_total <= 10000 THEN
        RETURN 'VIP';
    ELSE
        RETURN 'PREMIUM';
    END IF;

END;

/

--apel
```

SELECT categorie_client(1) FROM dual;

SELECT categorie_client(100) FROM dual;

```
CREATE OR REPLACE FUNCTION categorie_client (p_id IN CLIENTI_PRJ.id_client%TYPE)
RETURN VARCHAR2
AS
    v_exista NUMBER;
    v_total  NUMBER;
BEGIN
    SELECT COUNT(*)
    INTO v_exista
    FROM CLIENTI_PRJ
    WHERE id_client = p_id;

    IF v_exista = 0 THEN
        RETURN 'CLIENT INEXISTENT';
    END IF;

    SELECT NVL(SUM(pret_final), 0)
    INTO v_total
    FROM VANZARI_PRJ
    WHERE id_client = p_id;

    IF v_total < 5000 THEN
        RETURN 'NORMAL';
    ELSIF v_total <= 10000 THEN
        RETURN 'VIP';
    ELSE
        RETURN 'PREMIUM';
    END IF;
END;
/
--apel
SELECT categorie_client(1) FROM dual;
SELECT categorie_client(100) FROM dual;
```

Script Output x Query Result x Query Result 1 x

SQL | All Rows Fetched: 1 in 0,034 seconds

CATEGORIE_CLIENT(100)
1 CLIENT INEXISTENT

Script Output x Query Result x Query Result 1 x

SQL | All Rows Fetched: 1 in 0,061 seconds

CATEGORIE_CLIENT(1)
1 VIP

3) Să se construiască funcția *status_expozitie*, care primește ca parametru *ID_EXPOZITIE* și returnează statusul expoziției respective. Dacă data curentă se află între data deschiderii și data închiderii, funcția va returna *IN DESFASURARE*, altfel va returna *INCHEIATA*. Să se trateze cazul în care expoziția nu există. Cod:

```
CREATE OR REPLACE FUNCTION status_expozitie (p_id IN  
EXPOZITII_PRJ.id_expozitie%TYPE)
```

```
RETURN VARCHAR2
```

```
AS
```

```
    v_data_desch EXPOZITII_PRJ.data_deschidere%TYPE;
```

```
    v_data_inch EXPOZITII_PRJ.data_inchidere%TYPE;
```

```
BEGIN
```

```
    SELECT data_deschidere, data_inchidere
```

```
    INTO v_data_desch, v_data_inch
```

```
    FROM EXPOZITII_PRJ
```

```
    WHERE id_expozitie = p_id;
```

```
    IF SYSDATE BETWEEN v_data_desch AND v_data_inch THEN
```

```
        RETURN 'IN DESFASURARE';
```

```
    ELSE
```

```
        RETURN 'INCHEIATA';
```

```
    END IF;
```

```
EXCEPTION
```

```
    WHEN NO_DATA_FOUND THEN
```

```
        RETURN 'NU EXISTA';
```

```
END;
```

```
/
```

--apel1

SET SERVEROUTPUT ON;

SELECT id_expozitie, nume, status_expozitie(id_expozitie)

FROM EXPOZITII_PRJ;

/

--apel2

SET SERVEROUTPUT ON;

DECLARE

v_status VARCHAR2(50);

BEGIN

v_status := status_expozitie(&id_expozitie);

DBMS_OUTPUT.PUT_LINE('Status expozitie: ' || v_status);

END;

/

ID_EXPOZITIE	NUME	STATUS_EXPOZITIE(ID_EXPOZITIE)
1	1 Culori Urbane	INCHEIATA
2	2 Abstract si Forme	INCHEIATA
3	3 Portrete Contemporane	INCHEIATA
4	4 Lumi Imaginare	INCHEIATA
5	5 Minimalism si Linie	INCHEIATA
6	6 Peisaje si Natura	INCHEIATA

```
new:DECLARE
  v_status VARCHAR2(50);
BEGIN
  v_status := status_expozitie(3);
  DBMS_OUTPUT.PUT_LINE('Status expozitie: ' || v_status);
END;
Status expozitie: INCHEIATA
```

PL/SQL procedure successfully completed.

4) Să se construiască funcția *lucrare_max_artist*, care primește ca parametru *ID_ARTIST* și returnează titlul lucrării cu prețul estimativ maxim realizate de artistul respectiv. Să se trateze cazul în care artistul nu există sau nu are lucrări. Cod:

```
CREATE OR REPLACE FUNCTION lucrare_max_artist (p_id IN  
ARTISTI_PRJ.id_artist%TYPE)
```

```
RETURN VARCHAR2
```

```
AS
```

```
    v_exista NUMBER;
```

```
    v_titlu LUCRARI_ARTA_PRJ.titlu%TYPE;
```

```
BEGIN
```

```
    SELECT COUNT(*)
```

```
    INTO v_exista
```

```
    FROM ARTISTI_PRJ
```

```
    WHERE id_artist = p_id;
```

```
    IF v_exista = 0 THEN
```

```
        RETURN 'ARTIST INEXISTENT';
```

```
    END IF;
```

```
    SELECT titlu
```

```
    INTO v_titlu
```

```
    FROM LUCRARI_ARTA_PRJ
```

```
    WHERE id_artist = p_id
```

```
    AND pret_estimativ = (
```

```
        SELECT MAX(pret_estimativ)
```

```
        FROM LUCRARI_ARTA_PRJ
```

```

        WHERE id_artist = p_id
    )

    FETCH FIRST 1 ROW ONLY;

    RETURN v_titlu;

EXCEPTION

    WHEN NO_DATA_FOUND THEN

        RETURN 'Artistul nu are lucrari';

    WHEN OTHERS THEN

        RETURN 'A aparut o alta exceptie';

END;

/

--apel 1

SELECT &id_artist AS ID_ARTIST,

        lucrare_max_artist(&id_artist) AS LUCRARE_MAXIMA

FROM dual;

--apel 2

SELECT a.id_artist,

        a.nume,

        lucrare_max_artist(a.id_artist) AS LUCRARE_MAXIMA

FROM ARTISTI_PRJ a

WHERE a.id_artist = &id_artist;

```

Script Output x Query Result x Query Result 1 x		
SQL All Rows Fetched: 1 in 0,048 seconds		
ID_ARTIST	NUME	LUCRARE_MAXIMA
1	3 Luca Bianchi	Piazza veche

Script Output x Query Result x Query Result 1 x		
SQL All Rows Fetched: 1 in 0,065 seconds		
ID_ARTIST	LUCRARE_MAXIMA	
1	2 Piazza veche	

5) Să se construiască funcția *status_client_complet*, care primește ca parametru *ID_CLIENT* și returnează un șir de caractere ce conține numele clientului, numărul de achiziții, totalul cheltuit și categoria clientului. Să se trateze cazul în care clientul nu există. Cod:

```
CREATE OR REPLACE FUNCTION status_client_complet (p_id IN
CLIENTI_PRJ.id_client%TYPE)
```

```
RETURN VARCHAR2
```

```
AS
```

```
    v_nume CLIENTI_PRJ.nume%TYPE;
```

```
    v_nr    NUMBER;
```

```
    v_total NUMBER;
```

```
    v_cat   VARCHAR2(20);
```

```
BEGIN
```


SELECT nume

INTO v_nume

FROM CLIENTI_PRJ

WHERE id_client = p_id;

SELECT COUNT(*), NVL(SUM(pret_final), 0)

INTO v_nr, v_total

FROM VANZARI_PRJ

WHERE id_client = p_id;

IF v_total < 5000 THEN

v_cat := 'NORMAL';

ELSIF v_total <= 10000 THEN

v_cat := 'VIP';

ELSE

v_cat := 'PREMIUM';

END IF;

**RETURN v_nume || ' | achizitii: ' || v_nr || ' | total: ' || v_total || ' | categorie: ' ||
v_cat;**

EXCEPTION

WHEN NO_DATA_FOUND THEN

RETURN 'Nu exista clientul cu id-ul ' || p_id;

WHEN OTHERS THEN

RETURN 'A aparut o alta exceptie';

END;

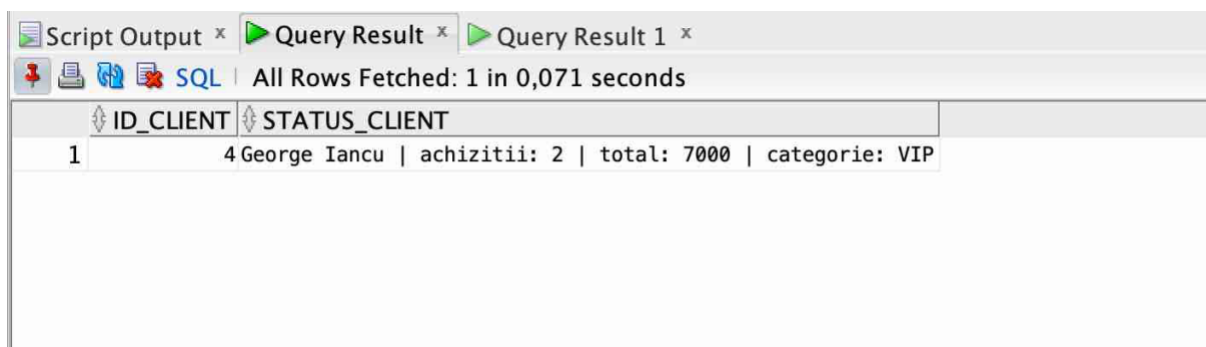
/

--apel 1

```
SELECT &id_client AS ID_CLIENT,  
        status_client_complet(&id_client) AS STATUS_CLIENT  
FROM dual;
```

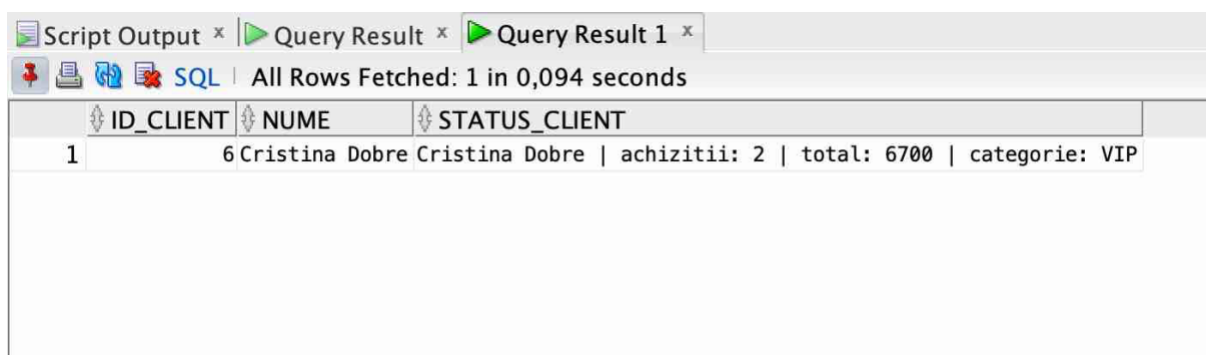
--apel 2

```
SELECT c.id_client,  
        c.nume,  
        status_client_complet(c.id_client) AS STATUS_CLIENT  
FROM CLIENTI_PRJ c  
WHERE c.id_client = &id_client;
```



The screenshot shows the SQL Developer interface with a query result window. The query has been executed, and the result is displayed in a table. The table has two columns: ID_CLIENT and STATUS_CLIENT. The first row shows ID_CLIENT 4 and STATUS_CLIENT '4 George Iancu | achizitii: 2 | total: 7000 | categorie: VIP'.

ID_CLIENT	STATUS_CLIENT
4	George Iancu achizitii: 2 total: 7000 categorie: VIP



The screenshot shows the SQL Developer interface with a query result window. The query has been executed, and the result is displayed in a table. The table has three columns: ID_CLIENT, NUME, and STATUS_CLIENT. The first row shows ID_CLIENT 6, NUME 'Cristina Dobre', and STATUS_CLIENT 'Cristina Dobre | achizitii: 2 | total: 6700 | categorie: VIP'.

ID_CLIENT	NUME	STATUS_CLIENT
6	Cristina Dobre	Cristina Dobre achizitii: 2 total: 6700 categorie: VIP

6) Să se construiască procedura *proc_raport_artist*, care primește ca parametru *ID_ARTIST* și afișează artistul, toate lucrările sale, prețul estimativ al fiecăreia, precum și valoarea totală estimativă și lucrarea cu prețul maxim. Să se trateze cazul în care artistul nu există sau nu are lucrări. Cod:

**CREATE OR REPLACE PROCEDURE proc_raport_artist (p_id IN
ARTISTI_PRJ.id_artist%TYPE)**

AS

CURSOR c_lucrari IS

SELECT id_lucrare, titlu, tehnica, pret_estimativ

FROM LUCRARI_ARTA_PRJ

WHERE id_artist = p_id

ORDER BY pret_estimativ DESC;

v_nume ARTISTI_PRJ.nume%TYPE;

v_tara ARTISTI_PRJ.tara_origine%TYPE;

v_id_luc LUCRARI_ARTA_PRJ.id_lucrare%TYPE;

v_titlu LUCRARI_ARTA_PRJ.titlu%TYPE;

v_tehn LUCRARI_ARTA_PRJ.tehnica%TYPE;

v_pret LUCRARI_ARTA_PRJ.pret_estimativ%TYPE;

v_total NUMBER := 0;

v_nr NUMBER := 0;

BEGIN

SELECT nume, tara_origine

INTO v_nume, v_tara

FROM ARTISTI_PRJ

WHERE id_artist = p_id;

DBMS_OUTPUT.PUT_LINE('ARTIST | ' || v_nume || ' | ' || v_tara);

**DBMS_OUTPUT.PUT_LINE('ID_LUCRARE | TITLU | TEHNICA |
PRET_ESTIMATIV');**

OPEN c_lucrari;

LOOP

FETCH c_lucrari INTO v_id_luc, v_titlu, v_tehn, v_pret;

EXIT WHEN c_lucrari%NOTFOUND;

**DBMS_OUTPUT.PUT_LINE(v_id_luc || ' ' || v_titlu || ' ' || v_tehn || ' ' ||
v_pret);**

v_total := v_total + v_pret;

v_nr := v_nr + 1;

END LOOP;

CLOSE c_lucrari;

IF v_nr = 0 THEN

DBMS_OUTPUT.PUT_LINE('Artistul exista, dar nu are lucrari.');

ELSE

DBMS_OUTPUT.PUT_LINE('TOTAL_ESTIMATIV | ' || v_total);

DBMS_OUTPUT.PUT_LINE('LUCRARE_MAX | ' || lucrare_max_artist(p_id));

END IF;

EXCEPTION

WHEN NO_DATA_FOUND THEN

DBMS_OUTPUT.PUT_LINE('Artistul nu exista.');

WHEN OTHERS THEN

```
DBMS_OUTPUT.PUT_LINE('A aparut o alta exceptie.');
```

```
END;
```

```
/
```

--apel 1

```
SET SERVEROUTPUT ON;
```

```
BEGIN
```

```
    proc_raport_artist(&id_artist);
```

```
END;
```

```
/
```

--apel 2

```
SET SERVEROUTPUT ON;
```

```
BEGIN
```

```
    DBMS_OUTPUT.PUT_LINE('ID_ARTIST | RAPORT');
```

```
    DBMS_OUTPUT.PUT_LINE(&id_artist || ' | START');
```

```
    proc_raport_artist(&id_artist);
```

```
END;
```

```
/
```

```
ARTIST | Luca Bianchi | Italia
ID_LUCRARE | TITLU | TEHNICA | PRET_ESTIMATIV
6 | Piazza veche | Ulei pe panza | 4000
7 | Portret italian | Ulei pe panza | 3990
23 | Portret batran | Ulei pe panza | 3100
TOTAL_ESTIMATIV | 11090
LUCRARE_MAX | Piazza veche
```

ID_ARTIST	RAPORT		
5	START		
ARTIST	Kenji Tanaka	Japonia	
ID_LUCRARE	TITLU	TEHNICA	PRET_ESTIMATIV
13	Fuziune traditionala	Ulei si tus	3800
12	Tokyo Night	Digital print	2750
TOTAL_ESTIMATIV	6550		
LUCRARE_MAX	Fuziune traditionala		

7) Să se construiască procedura *proc_actualizeaza_expozitie*, care primește ca parametru *ID_EXPOZITIE* și actualizează locul în expoziție pentru toate lucrările care nu au prefixul *SECTOR-*, apoi afișează lucrările modificate și valoarea totală estimativă a expoziției. Cod:

```
CREATE OR REPLACE PROCEDURE proc_actualizeaza_expozitie (p_id IN
EXPOZITII_PRJ.id_expozitie%TYPE)
```

```
AS
```

```
CURSOR c_lucrari IS
```

```
SELECT le.id_lucrare, l.titlu, le.loc_in_expozitie, l.pret_estimativ
```

```
FROM LUCRARI_EXPOZITII_PRJ le, LUCRARI_ARTA_PRJ l
```

```
WHERE le.id_lucrare = l.id_lucrare
```

```
AND le.id_expozitie = p_id
```

```
ORDER BY le.id_lucrare;
```

```
v_id_luc LUCRARI_ARTA_PRJ.id_lucrare%TYPE;
```

```
v_titlu LUCRARI_ARTA_PRJ.titlu%TYPE;
```

```
v_loc LUCRARI_EXPOZITII_PRJ.loc_in_expozitie%TYPE;
```

```
v_pret LUCRARI_ARTA_PRJ.pret_estimativ%TYPE;
```

```
v_total NUMBER := 0;
```

```
v_exista NUMBER;
```

```
BEGIN
```

```
SELECT COUNT(*)
```

```

INTO v_exista

FROM EXPOZITII_PRJ

WHERE id_expozitie = p_id;

IF v_exista = 0 THEN

    DBMS_OUTPUT.PUT_LINE('Expozitia nu exista.');

ELSE

    UPDATE LUCRARI_EXPOZITII_PRJ

    SET loc_in_expozitie = 'SECTOR-' || loc_in_expozitie

    WHERE id_expozitie = p_id

    AND loc_in_expozitie NOT LIKE 'SECTOR-%';

    DBMS_OUTPUT.PUT_LINE('RANDURI_MODIFICATE | ' ||
SQL%ROWCOUNT);

    DBMS_OUTPUT.PUT_LINE('ID_LUCRARE | TITLU | LOC |
PRET_ESTIMATIV');

    OPEN c_lucrari;

    LOOP

        FETCH c_lucrari INTO v_id_luc, v_titlu, v_loc, v_pret;

        EXIT WHEN c_lucrari%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(v_id_luc || ' | ' || v_titlu || ' | ' || v_loc || ' | ' ||
v_pret);

        v_total := v_total + v_pret;

    END LOOP;

```

CLOSE c_lucrari;

DBMS_OUTPUT.PUT_LINE('TOTAL_EXPOZITIE | ' || v_total);

COMMIT;

END IF;

EXCEPTION

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('A aparut o alta exceptie.');

END;

/

--apel 1

SET SERVEROUTPUT ON;

BEGIN

proc_actualizeaza_expozitie(&id_expozitie);

END;

/

--apel 2

SET SERVEROUTPUT ON;

BEGIN

DBMS_OUTPUT.PUT_LINE('ID_EXPOZITIE | ACTUALIZARE');


```
DBMS_OUTPUT.PUT_LINE(&id_expozitie || ' | START');
```

```
proc_actualizeaza_expozitie(&id_expozitie);
```

```
END;
```

```
/
```

```
RANDURI_MODIFICATE | 0
ID_LUCRARE | TITLU | LOC | PRET_ESTIMATIV
3 | Strada veche | SECTOR-Sala 3 intrare | 3000
7 | Portret italian | SECTOR-Sala 1 portrete A | 3990
10 | Portret de femeie | SECTOR-Sala 1 portrete B | 3500
11 | Familie in parc | SECTOR-Sala 2 perete portrete | 3300
23 | Portret batran | SECTOR-Sala 2 colt clasic | 3100
TOTAL_EXPOZITIE | 16890
```

```
ID_EXPOZITIE | ACTUALIZARE
4 | START
Expozitia nu exista.
```

8) Să se construiască procedura *proc_raport_client*, care primește ca parametru *ID_CLIENT* și afișează toate vânzările clientului, titlul lucrării, data vânzării, prețul final, totalul cheltuit și lucrarea cumpărată la prețul cel mai mare. Să se trateze cazul în care clientul nu există. Cod:

```
CREATE OR REPLACE FUNCTION lucrare_scumpa_client (p_id IN
CLIENTI_PRJ.id_client%TYPE)
```

```
RETURN VARCHAR2
```

```
AS
```

```
v_titlu LUCRARI_ARTA_PRJ.titlu%TYPE;
```

```
BEGIN
```

```
SELECT l.titlu
```

```
INTO v_titlu
```

```
FROM VANZARI_PRJ v, LUCRARI_ARTA_PRJ l
```

```
WHERE v.id_lucrare = l.id_lucrare
```

```
AND v.id_client = p_id
```

```
AND v.pret_final = (
```

```

        SELECT MAX(pret_final)

        FROM VANZARI_PRJ

        WHERE id_client = p_id

    )

    FETCH FIRST 1 ROW ONLY;

    RETURN v_titlu;

EXCEPTION

    WHEN NO_DATA_FOUND THEN

        RETURN 'Client fara achizitii';

    WHEN OTHERS THEN

        RETURN 'Eroare';

END;

/

CREATE OR REPLACE PROCEDURE proc_raport_client (p_id IN
CLIENTI_PRJ.id_client%TYPE)

AS

    CURSOR c_vanzari IS

        SELECT v.id_vanzare, l.titlu, v.data_vanzare, v.pret_final

        FROM VANZARI_PRJ v, LUCRARI_ARTA_PRJ l

        WHERE v.id_lucrare = l.id_lucrare

        AND v.id_client = p_id

        ORDER BY v.data_vanzare;

    v_nume CLIENTI_PRJ.nume%TYPE;

```

v_id_vanzare VANZARI_PRJ.id_vanzare%TYPE;

v_titlu LUCRARI_ARTA_PRJ.titlu%TYPE;

v_data VANZARI_PRJ.data_vanzare%TYPE;

v_pret VANZARI_PRJ.pret_final%TYPE;

v_nr NUMBER := 0;

v_total NUMBER := 0;

BEGIN

SELECT nume

INTO v_nume

FROM CLIENTI_PRJ

WHERE id_client = p_id;

DBMS_OUTPUT.PUT_LINE('CLIENT | ' || v_nume);

*DBMS_OUTPUT.PUT_LINE('ID_VANZARE | TITLU | DATA_VANZARE |
PRET_FINAL');*

OPEN c_vanzari;

LOOP

FETCH c_vanzari INTO v_id_vanzare, v_titlu, v_data, v_pret;

EXIT WHEN c_vanzari%NOTFOUND;

*DBMS_OUTPUT.PUT_LINE(v_id_vanzare || ' | ' || v_titlu || ' | ' ||
TO_CHAR(v_data, 'DD-MM-YYYY') || ' | ' || v_pret);*

v_nr := v_nr + 1;

v_total := v_total + v_pret;

END LOOP;

CLOSE c_vanzari;

DBMS_OUTPUT.PUT_LINE('TOTAL_CHELTUIT | ' || v_total);

**DBMS_OUTPUT.PUT_LINE('TOP_LUCRARE | ' ||
lucrare_scumpa_client(p_id));**

EXCEPTION

WHEN NO_DATA_FOUND THEN

DBMS_OUTPUT.PUT_LINE('Clientul nu exista.');

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('A aparut o alta exceptie.');

END;

/

--apel 1

SET SERVEROUTPUT ON;

BEGIN

proc_raport_client(&id_client);

END;

/

--apel 2

SET SERVEROUTPUT ON;

BEGIN

DBMS_OUTPUT.PUT_LINE('ID_CLIENT | RAPORT');

DBMS_OUTPUT.PUT_LINE(&id_client || ' | START');

proc_raport_client(&id_client);

END;

/

```
CLIENT | Ion Radu
ID_VANZARE | TITLU | DATA_VANZARE | PRET_FINAL
1 | Noapte in Bucuresti | 15-03-2022 | 3600
21 | Umbre in oras | 20-08-2022 | 3650
TOTAL_CHELTUIT | 7250
TOP_LUCRARE | Umbre in oras
```

```
-----
ID_CLIENT | RAPORT
3 | START
CLIENT | Laura Pop
ID_VANZARE | TITLU | DATA_VANZARE | PRET_FINAL
4 | Abstractie albastra | 10-04-2022 | 2700
24 | Podul peste Sena | 12-09-2022 | 2400
TOTAL_CHELTUIT | 5100
TOP_LUCRARE | Abstractie albastra
```

PL/SQL procedure successfully completed.

9) Să se construiască procedura *proc_dubleaza_lucrari_artist*, care primește ca parametru *ID_ARTIST*, dublează prețul estimativ al lucrărilor artistului respectiv și memorează titlurile acestora într-o colecție de tip tabelă indexată. La final să se afișeze elementele colecției și totalul lucrărilor modificate. Cod:

CREATE OR REPLACE PROCEDURE *proc_dubleaza_lucrari_artist* (p_id IN ARTISTI_PRJ.id_artist%TYPE)

AS

TYPE *colectie* IS TABLE OF LUCRARI_ARTA_PRJ.titlu%TYPE INDEX BY PLS_INTEGER;

v_titluri *colectie*;

```

i NUMBER := 0;

v_exista NUMBER;

BEGIN

SELECT COUNT(*)
INTO v_exista
FROM ARTISTI_PRJ
WHERE id_artist = p_id;

IF v_exista = 0 THEN

    DBMS_OUTPUT.PUT_LINE('Artistul nu exista.');
```

```

ELSE

    FOR r IN (

        SELECT id_lucrare, titlu

        FROM LUCRARI_ARTA_PRJ

        WHERE id_artist = p_id

        ORDER BY id_lucrare

    ) LOOP

        UPDATE LUCRARI_ARTA_PRJ

        SET pret_estimativ = pret_estimativ * 2

        WHERE id_lucrare = r.id_lucrare;

        i := i + 1;

        v_titluri(i) := r.titlu;

    END LOOP;

```

```

DBMS_OUTPUT.PUT_LINE('TOTAL_LUCRARI_MODIFICATE | ' || i);

IF i > 0 THEN

    FOR j IN v_titluri.FIRST .. v_titluri.LAST LOOP

        DBMS_OUTPUT.PUT_LINE('Element ' || j || ': ' || v_titluri(j));

    END LOOP;

    COMMIT;

ELSE

    DBMS_OUTPUT.PUT_LINE('Artistul exista, dar nu are lucrari.');
```

END IF;

END IF;

EXCEPTION

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('A aparut o alta exceptie.');

END;

/

--apel 1

SET SERVEROUTPUT ON;

BEGIN

proc_dubleaza_lucrari_artist(&id_artist);

END;

/

--apel 2

SET SERVEROUTPUT ON;

BEGIN

DBMS_OUTPUT.PUT_LINE('ID_ARTIST | DUBLARE');

DBMS_OUTPUT.PUT_LINE(&id_artist || ' | START');

proc_dubleaza_lucrari_artist(&id_artist);

END;

/

```
TOTAL_LUCRARI_MODIFICATE | 3
Element 1: Linii de Paris
Element 2: Minimal Paris
Element 3: Podul peste Sena
```

```
ID_ARTIST | DUBLARE
5 | START
TOTAL_LUCRARI_MODIFICATE | 2
Element 1: Portret de femeie
Element 2: Familie in parc
```

```
PL/SQL procedure successfully completed.
```

Triggeri:

1)Să se creeze un trigger care să nu permită inserarea unei lucrări dacă anul realizării este în viitor.

CREATE OR REPLACE TRIGGER trg_an_realizare

BEFORE INSERT OR UPDATE ON LUCRARI_ARTA_PRJ

FOR EACH ROW

BEGIN

IF :NEW.an_realizare > 2026 THEN


```
        RAISE_APPLICATION_ERROR(-20001, 'Anul realizarii nu poate fi in viitor.');
```

```
    END IF;
```

```
END;
```

```
/
```

```
Trigger TRG_AN_REALIZARE compiled
```

2) Să se creeze un trigger care să verifice prețul final al unei vânzări să nu fie 0 sau negativ.

```
CREATE OR REPLACE TRIGGER trg_pret_pozitiv
```

```
BEFORE INSERT ON VANZARI_PRJ
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    IF :NEW.pret_final <= 0 THEN
```

```
        RAISE_APPLICATION_ERROR(-20002, 'Pretul trebuie sa fie mai mare decat 0.');
```

```
    END IF;
```

```
END;
```

```
/
```

```
Trigger TRG_PRET_POZITIV compiled
```

3) Să se creeze un trigger care, la ștergerea unui artist, să afișeze un mesaj simplu (fără tabele de audit complicate).

```
CREATE OR REPLACE TRIGGER trg_mesaj_stergere
```

```
AFTER DELETE ON ARTISTI_PRJ
```

```
FOR EACH ROW
```

BEGIN

DBMS_OUTPUT.PUT_LINE('Artistul cu ID ' || :OLD.id_artist || ' a fost eliminat.');

END;

/

Trigger TRG_MESAJ_STERGERE compiled

4) Să se creeze un trigger care să pună automat data curentă la o vânzare dacă aceasta este introdusă ca fiind nulă.

CREATE OR REPLACE TRIGGER trg_data_vanzare

BEFORE INSERT ON VANZARI_PRJ

FOR EACH ROW

BEGIN

IF :NEW.data_vanzare IS NULL THEN

:NEW.data_vanzare := SYSDATE;

END IF;

END;

/

Trigger TRG_DATA_VANZARE compiled

5) Să se creeze un trigger care să limiteze prețul estimativ maxim al unei lucrări la 100.000.

CREATE OR REPLACE TRIGGER trg_limita_pret

BEFORE INSERT OR UPDATE OF pret_estimativ ON LUCRARI_ARTA_PRJ

FOR EACH ROW

BEGIN

IF :NEW.pret_estimativ > 100000 THEN

```
RAISE_APPLICATION_ERROR(-20003, 'Pretul depaseste limita maxima permisa.');
```

```
END IF;
```

```
END;
```

```
/
```

```
Trigger TRG_LIMITA_PRET compiled
```

6) Să se creeze un trigger care să transforme automat tehnica lucrării în litere mari (UPPER).

```
CREATE OR REPLACE TRIGGER trg_tehnica_upper
```

```
BEFORE INSERT OR UPDATE ON LUCRARI_ARTA_PRJ
```

```
FOR EACH ROW
```

```
BEGIN
```

```
:NEW.tehnica := UPPER(:NEW.tehnica);
```

```
END;
```

```
/
```

```
Trigger TRG_TEHNICA_UPPER compiled
```

Pachete:

1) Să se creeze un pachet care folosește supraîncărcarea pentru a căuta un artist (după ID sau după Nume) și care utilizează o colecție pentru a stoca titlurile lucrărilor înainte de a le afișa.

```
CREATE OR REPLACE PACKAGE PAC_ARTISTI_ADV AS
```

```
FUNCTION cauta_artist(p_id NUMBER) RETURN VARCHAR2;
```

```
FUNCTION cauta_artist(p_nume VARCHAR2) RETURN NUMBER;
```

```
PROCEDURE colecteaza_si_afiseaza(p_id_artist NUMBER);
```

END PAC_ARTISTI_ADV;

/

--(corpul)

CREATE OR REPLACE PACKAGE BODY PAC_ARTISTI_ADV AS

FUNCTION caută_artist(p_id NUMBER) RETURN VARCHAR2 IS

v_nume ARTISTI_PRJ.nume%TYPE;

BEGIN

SELECT nume INTO v_nume FROM ARTISTI_PRJ WHERE id_artist = p_id;

RETURN v_nume;

EXCEPTION WHEN NO_DATA_FOUND THEN RETURN 'Negăsit';

END caută_artist;

FUNCTION caută_artist(p_nume VARCHAR2) RETURN NUMBER IS

v_id NUMBER;

BEGIN

**SELECT id_artist INTO v_id FROM ARTISTI_PRJ WHERE UPPER(nume) =
UPPER(p_nume);**

RETURN v_id;

EXCEPTION WHEN NO_DATA_FOUND THEN RETURN -1;

END caută_artist;

PROCEDURE colectează_si_afisează(p_id_artist NUMBER) IS

**TYPE t_lista_titluri IS TABLE OF LUCRARI_ARTA_PRJ.titlu%TYPE INDEX
BY PLS_INTEGER;**

v_lucrari t_lista_titluri;

```

    v_idx NUMBER := 0;

BEGIN

    FOR r IN (SELECT titlu FROM LUCRARI_ARTA_PRJ WHERE id_artist =
p_id_artist) LOOP

        v_idx := v_idx + 1;

        v_lucrari(v_idx) := r.titlu;

    END LOOP;

    IF v_lucrari.COUNT > 0 THEN

        FOR i IN 1..v_lucrari.COUNT LOOP

            DBMS_OUTPUT.PUT_LINE('Lucrarea ' || i || ': ' || v_lucrari(i));

        END LOOP;

    ELSE

        DBMS_OUTPUT.PUT_LINE('Artistul nu are lucrări.');

```

```
Package PAC_ARTISTI_ADV compiled
```

```
Package Body PAC_ARTISTI_ADV compiled
```

```
Nume artist ID 1: Andrei Popescu
```

```
Lucrarea 1: Noapte in Bucuresti
```

```
Lucrarea 2: Blocuri de lumina
```

```
Lucrarea 3: Strada veche
```

```
PL/SQL procedure successfully completed.
```

2) Să se creeze un pachet care utilizează o variabilă de pachet pentru a menține un "prag de profit" și o funcție care determină dacă o vânzare este "profitabilă" comparând prețul final cu cel estimativ.

```
CREATE OR REPLACE PACKAGE PAC_FINANCIAR AS
```

```
    g_comision_galerie NUMBER := 0.10;
```

```
    FUNCTION este_profitabila(p_id_vanzare NUMBER) RETURN VARCHAR2;
```

```
    PROCEDURE actualizeaza_comision(p_nou_comision NUMBER);
```

```
END PAC_FINANCIAR;
```

```
/
```

```
--(corpul)
```

```
CREATE OR REPLACE PACKAGE BODY PAC_FINANCIAR AS
```

```
    FUNCTION este_profitabila(p_id_vanzare NUMBER) RETURN VARCHAR2 IS
```

```
        v_pret_final NUMBER;
```

```
        v_pret_est  NUMBER;
```

```
BEGIN
```

```

SELECT v.pret_final, l.pret_estimativ

INTO v_pret_final, v_pret_est

FROM VANZARI_PRJ v JOIN LUCRARI_ARTA_PRJ l ON v.id_lucrare =
l.id_lucrare

WHERE v.id_vanzare = p_id_vanzare;

IF v_pret_final > (v_pret_est * (1 + g_comision_galerie)) THEN

    RETURN 'PROFITABILĂ';

ELSE

    RETURN 'MARGINE MICĂ';

END IF;

END este_profitabila;

PROCEDURE actualizeaza_comision(p_nou_comision NUMBER) IS

BEGIN

    g_comision_galerie := p_nou_comision;

    DBMS_OUTPUT.PUT_LINE('Noul comision setat: ' || (g_comision_galerie *
100) || '%');

    END actualizeaza_comision;

END PAC_FINANCIAR;

/

SET SERVEROUTPUT ON;

BEGIN

    DBMS_OUTPUT.PUT_LINE('Status profit vânzare ID 1: ' ||
PAC_FINANCIAR.este_profitabila(1));

    PAC_FINANCIAR.actualizeaza_comision(0.20);

```

```
DBMS_OUTPUT.PUT_LINE('Status după mărire comision: ' ||  
PAC_FINANCIAR.este_profitabila(1));
```

```
END;
```

```
/
```

```
Package PAC_FINANCIAR compiled
```

```
Package Body PAC_FINANCIAR compiled
```

```
Status profit vânzare ID 1: MARGINE MICĂ
```

```
Noul comision setat: 20%
```

```
Status după mărire comision: MARGINE MICĂ
```

```
PL/SQL procedure successfully completed.
```

3) Să se construiască o procedură avansată care validează capacitatea unei locații înainte de a muta toate lucrările dintr-o expoziție în alta, folosind tranzacții.

--(pachet)

```
CREATE OR REPLACE PACKAGE PAC_LOGISTICA_EXPO AS
```

```
    PROCEDURE mutare_securizata(p_id_sursa NUMBER, p_id_dest NUMBER);
```

```
END PAC_LOGISTICA_EXPO;
```

```
/
```

--(copr)

```
CREATE OR REPLACE PACKAGE BODY PAC_LOGISTICA_EXPO AS
```

```
    PROCEDURE mutare_securizata(p_id_sursa NUMBER, p_id_dest NUMBER)
```

```
IS
```

```
    v_nr_lucrari_sursa NUMBER;
```

```
    v_nr_lucrari_dest NUMBER;
```



```

e_capacitate_depasita EXCEPTION;

BEGIN

    SAVEPOINT punct_control;

    SELECT COUNT(*) INTO v_nr_lucrari_sursa FROM
LUCRARI_EXPOZITII_PRJ WHERE id_expozitie = p_id_sursa;

    SELECT COUNT(*) INTO v_nr_lucrari_dest FROM
LUCRARI_EXPOZITII_PRJ WHERE id_expozitie = p_id_dest;

    IF (v_nr_lucrari_sursa + v_nr_lucrari_dest) > 15 THEN

        RAISE e_capacitate_depasita;

    END IF;

    UPDATE LUCRARI_EXPOZITII_PRJ SET id_expozitie = p_id_dest WHERE
id_expozitie = p_id_sursa;

    DBMS_OUTPUT.PUT_LINE('Mutare finalizată cu succes.');
```

COMMIT;

```

EXCEPTION

    WHEN e_capacitate_depasita THEN

        ROLLBACK TO punct_control;

        DBMS_OUTPUT.PUT_LINE('Eroare: Capacitate insuficientă în expoziția
destinație!');
```

WHEN OTHERS THEN

```

        ROLLBACK TO punct_control;

        DBMS_OUTPUT.PUT_LINE('Eroare neprevăzută.');
```

END mutare_securizata;

END PAC_LOGISTICA_EXPO;

```

/
SET SERVEROUTPUT ON;
BEGIN
    PAC_LOGISTICA_EXPO.mutare_securizata(1, 2);

    DBMS_OUTPUT.PUT_LINE('Procedura de mutare a fost executată.');
```

END;

/

```

Package PAC_LOGISTICA_EXPO compiled

Package Body PAC_LOGISTICA_EXPO compiled

Eroare neprevăzută.
Procedura de mutare a fost executată.

PL/SQL procedure successfully completed.
```

4) Să se construiască un pachet care să conțină o funcție pentru generarea unui raport structurat de activitate pentru un client (afișând numele și totalul cheltuit sub formă de text formatat) și o funcție care să identifice prin tehnici de filtrare, câți artiști din baza de date nu au încă nicio lucrare înregistrată.

--(pachet)

```

CREATE OR REPLACE PACKAGE PAC_SISTEM_AUDIT AS

    FUNCTION genereaza_raport_client(p_id_client NUMBER) RETURN
    VARCHAR2;

    FUNCTION artist_fara_lucrari RETURN NUMBER;

END PAC_SISTEM_AUDIT;

/
```

--(corp)

```

CREATE OR REPLACE PACKAGE BODY PAC_SISTEM_AUDIT AS

    FUNCTION genereaza_raport_client(p_id_client NUMBER) RETURN
    VARCHAR2 IS

        v_info VARCHAR2(1000);

        v_nume CLIENTI_PRJ.nume%TYPE;

        v_total NUMBER;

    BEGIN

        SELECT nume INTO v_nume FROM CLIENTI_PRJ WHERE id_client =
        p_id_client;

        SELECT NVL(SUM(pret_final), 0) INTO v_total FROM VANZARI_PRJ
        WHERE id_client = p_id_client;

        v_info := '{ "client": ' || v_nume || ', "total_cheltuit": ' || v_total || ' }';

        RETURN v_info;

    EXCEPTION WHEN NO_DATA_FOUND THEN RETURN '{"error": "ID
    invalid"}';

    END genereaza_raport_client;


    FUNCTION artist_fara_lucrari RETURN NUMBER IS

        v_contor NUMBER;

    BEGIN

        SELECT COUNT(*) INTO v_contor

        FROM ARTISTI_PRJ a

        WHERE NOT EXISTS (SELECT 1 FROM LUCRARI_ARTA_PRJ l WHERE
        l.id_artist = a.id_artist);

        RETURN v_contor;

    END artist_fara_lucrari;

END PAC_SISTEM_AUDIT;

/

```

SET SERVEROUTPUT ON;

DECLARE

v_raport VARCHAR2(1000);

BEGIN

v_raport := PAC_SISTEM_AUDIT.genereaza_raport_client(1);

DBMS_OUTPUT.PUT_LINE('Raport Client: ' || v_raport);

**DBMS_OUTPUT.PUT_LINE('Număr artiști fără nicio lucrare: ' ||
PAC_SISTEM_AUDIT.artist_fara_lucrari);**

END;

/

Package PAC_SISTEM_AUDIT compiled

Package Body PAC_SISTEM_AUDIT compiled

Raport Client: { "client": "Ion Radu", "total_cheltuit": 7250 }
Număr artiști fără nicio lucrare: 0

PL/SQL procedure successfully completed.